安恒信息
DAS-security 安全中国



人工智能基础概念和技术

杭州安恒信息技术股份有限公司 陈凯

www.dbappsecurity.com.cn

章节	学习目标
基础算法原理	■ 机器学习基本概述 ■ 机器学习三个基本要素
深度神经网络	■ 深度学习模型（DNN）结构 ■ DNN 前向传播 ■ DNN 反向传播
环境准备	■ Pytorch 安装

安恒信息

Part 01

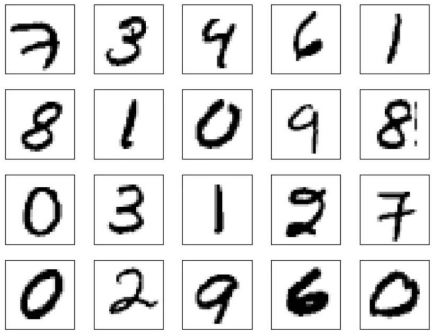
基础算法原理

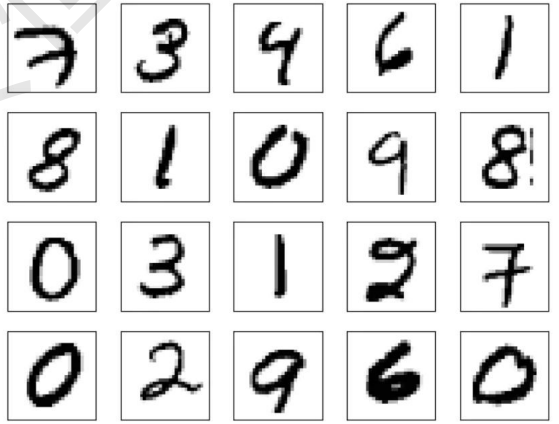
机器学习技术

通俗的讲，机器学习就是让计算机从数据中进行自动学习，得到某种知识（或规律）。作为一门学科，机器学习通常指如何从观测数据（样本）中寻找规律，并利用学习到的规律（模型）对未知或者无法观测数据进行预测，这类问题的解决方法。

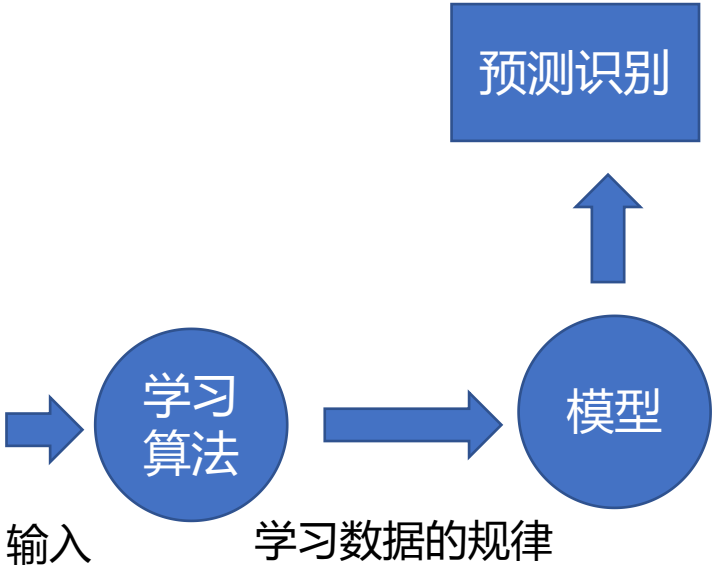








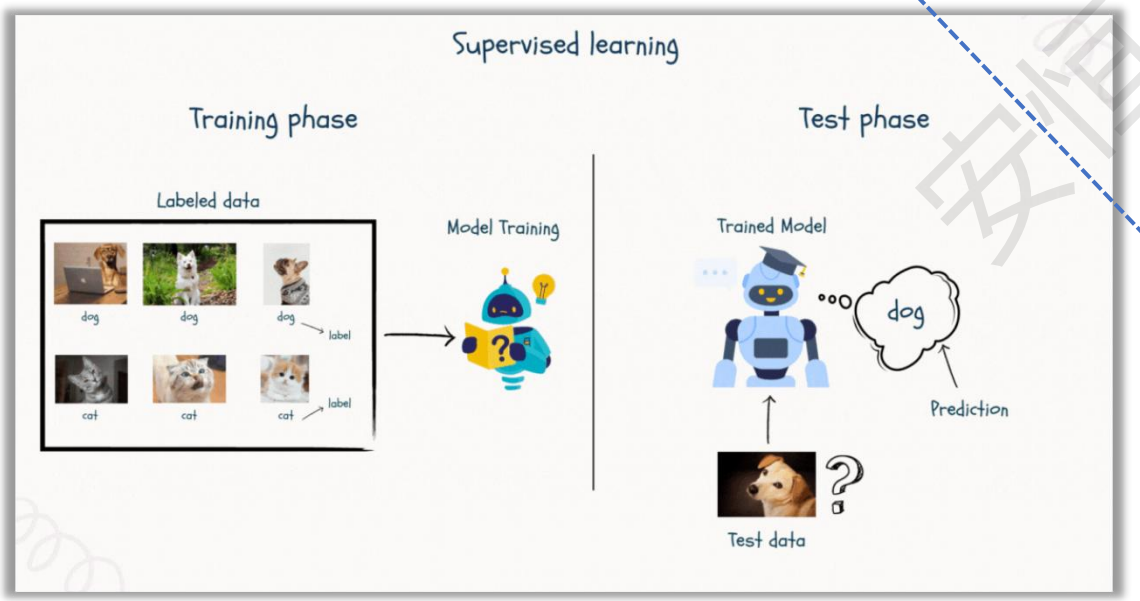
手写数字识别数据集（MNIST数据集）

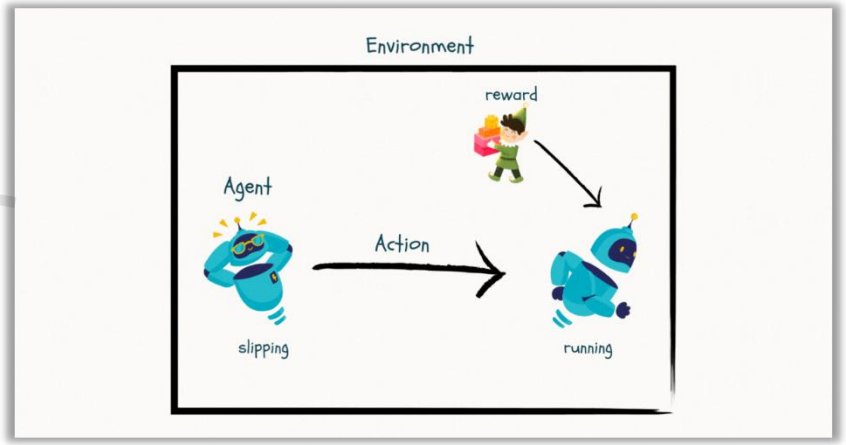


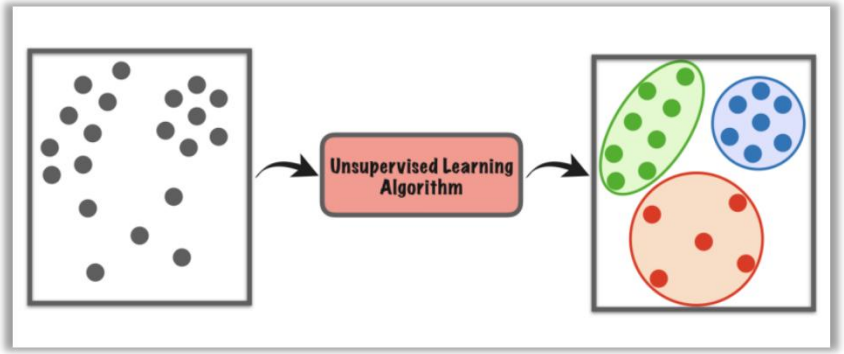
安恒信息

基本分类

机器学习一般包括**监督学习**、**无监督学习**、**强化学习**。







监督学习

监督学习是从标注数据中学习预测模型的机器学习问题，标注数据表示输入输出的对应关系，预测模型对给定输入产生相应的输出。**监督学习的本质是学习输入到输出的映射的统计规律**。

判断一个西瓜是不是 好/甜 西瓜

离散变量　连续变量

输入样本数据: X

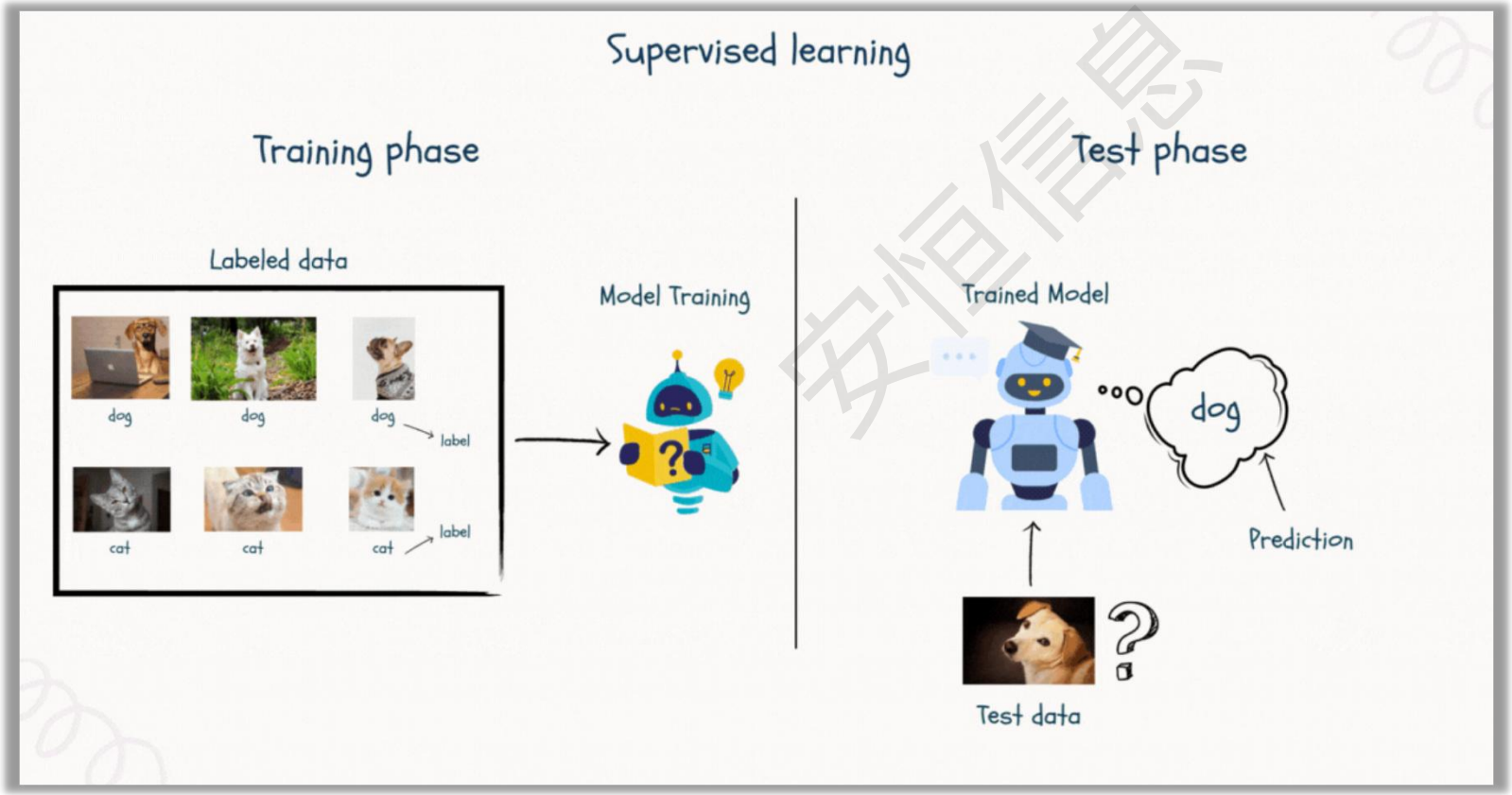
编号	色泽	敲声	根蒂	好瓜/甜度
1	青绿	浊响	蜷缩	是/0.8
2	青绿	清脆	硬挺	否/0.3
3	乌黑	浊响	蜷缩	是/0.9
4	乌黑	沉闷	稍卷	否/0.2



索引　　特征: $\{x_1, x_2, x_3\}$　　标签值: y

监督学习

监督学习：就是已知数据特征和结果的情况下，学习数据中的规律，最后达到能预测和总结的能力。



机器学习的三个基本要素

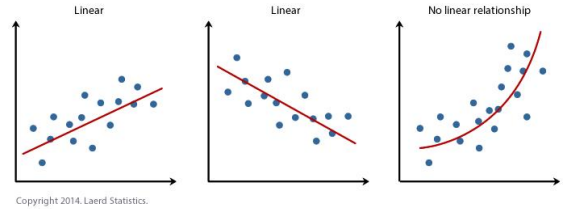
机器学习是从有限的观测数据或（猜测）出具有一般性的规律，并可与将总结出来的规律推广到观测样本上。机器学习的方法可以粗略的分为三个基本要素，模型，学习准则，优化算法 。



安恒信息

模型可以理解成是函数，这个函数可能简单（线性的函数），可能复杂（非线性的函数）

模型



编号	青绿程度	浊响程度	蜷缩程度	甜度
1	0.8	0.8	0.7	0.8

$$\text{甜度 } f(x) = \text{权重}_1 * 0.8 + \text{权重}_2 * 0.8 + \text{权重}_3 * 0.7 \approx 0.8$$

假设现在我们初始化一个线性函数

$$\text{函数 1：} \quad f(x) = 1 * 0.8 + 0 * 0.8 + 0 * 0.7 = 0.8$$

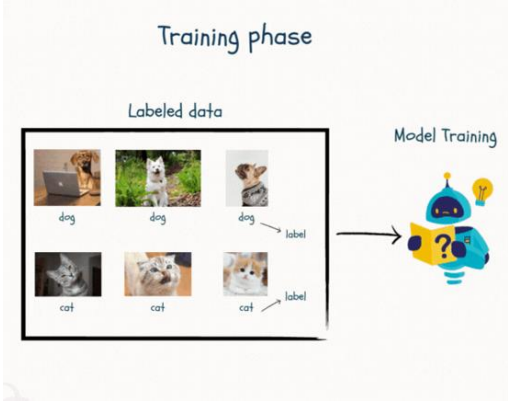
$$\text{函数 2：} \quad f(x) = 0.3 * 0.8 + 0.6 * 0.8 + 0.1 * 0.7 \approx 0.79$$

目标是找到这个函数在数据下表现最好的参数（找参数的过程就是训练）

编号	青绿程度	浊响程度	蜷缩程度	甜度
2	0.6	0.9	0.8	0.8

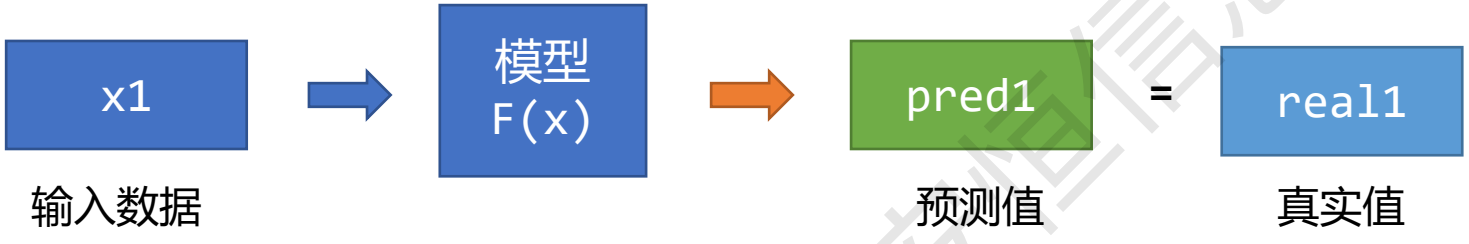
$$\text{函数 1：} \quad f(x) = 1 * 0.6 + 0 * 0.9 + 0 * 0.8 = 0.6$$

$$\text{函数 2：} \quad f(x) = 0.3 * 0.6 + 0.6 * 0.9 + 0.1 * 0.8 = 0.8$$



模型

机器学习的目标就是建立出这样一个模型：在数据输入之后模型得到的结果和真实环境中的结果尽可能相近。

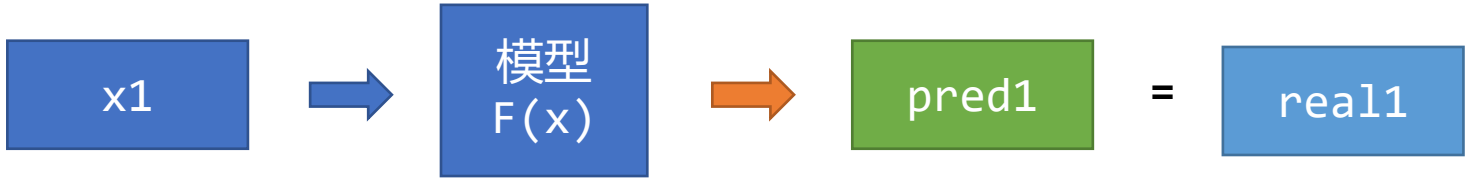


模型 = 函数

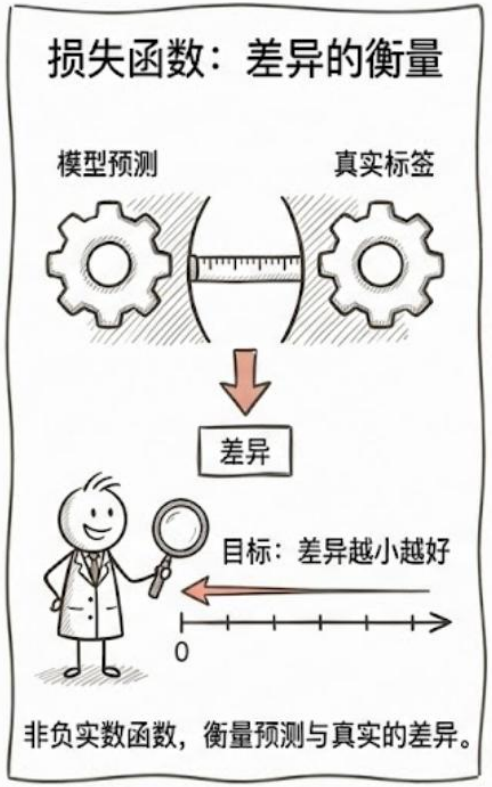
安恒信息

机器学习三个基本要素

学习准则



输入数据　　**模型 = 函数**　　预测值　　真实值



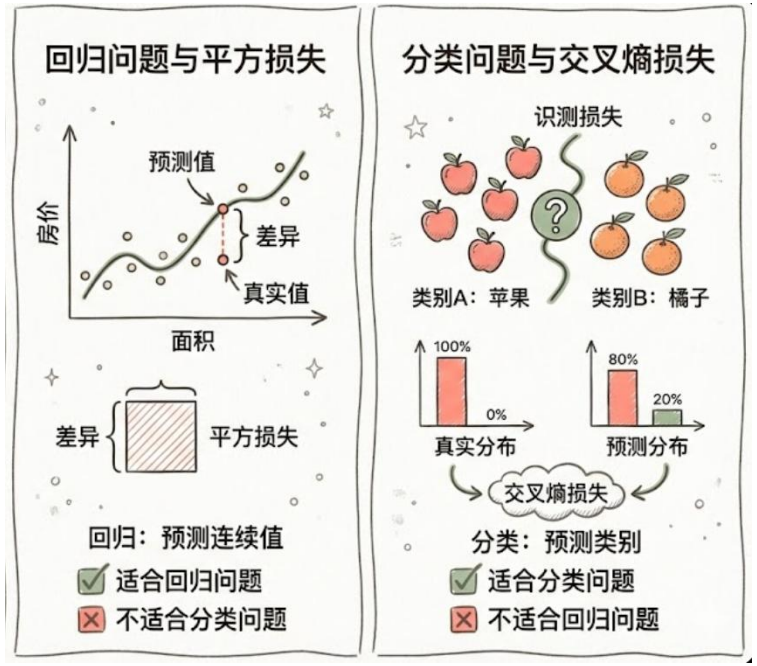
损失函数是一个非负实数函数，用来衡量模型预测和真实标签之间的差异，常用的损失函数有：

- 1、平方损失函数：

$$L(y, f(x;\theta)) = \frac{1}{2}(y - f(x;\theta))^2$$

- 2、交叉熵损失函数：

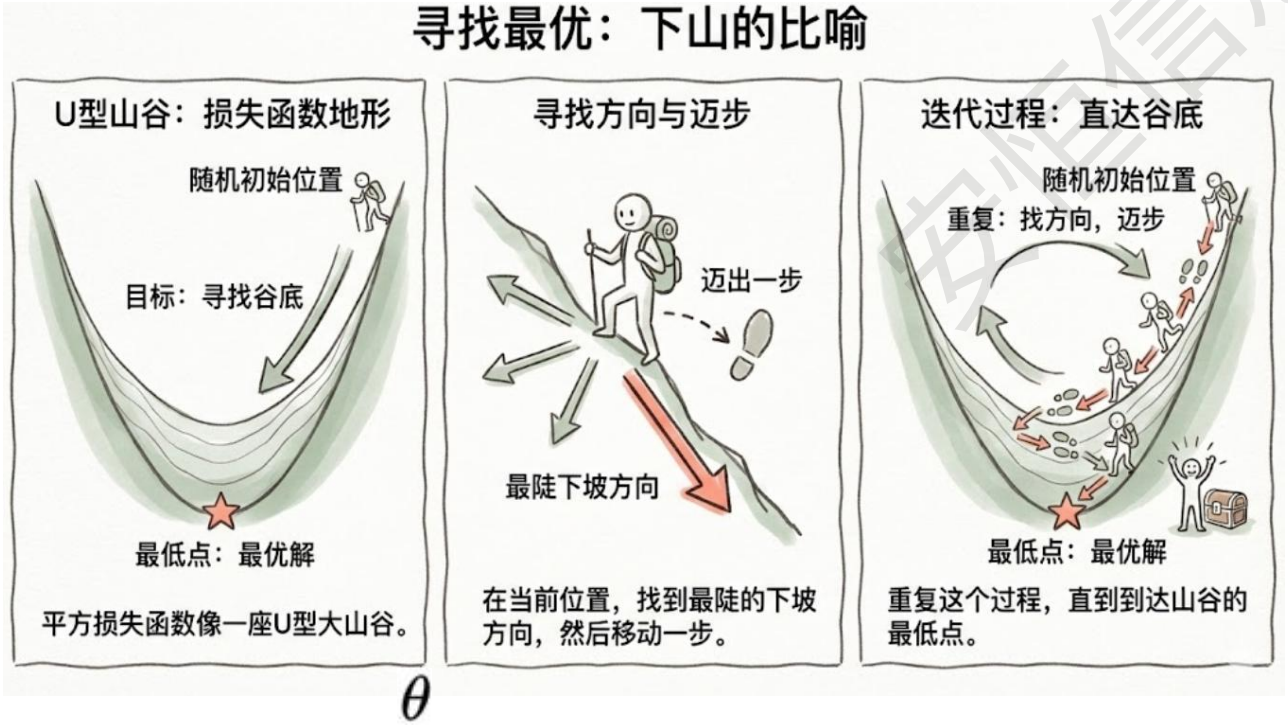
$$L(y, f(x;\theta)) = -y^T log f(x;\theta) = \sum_{c=1}^{C} y_c log f_c(x;\theta)$$



安恒信息

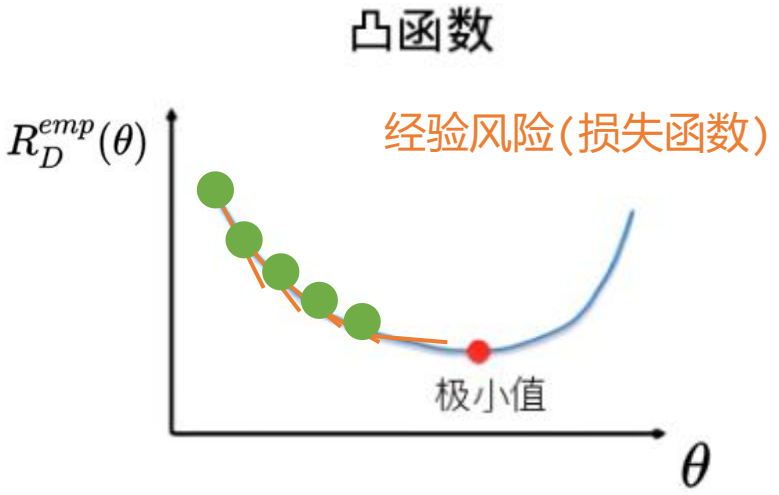
优化方法：梯度下降

确定了训练数据，学习准则，如何找到最优模型f(x,θ*)拟合数据就成为了一个最优化问题。



以优化模型经验风险为例：

$$R_D^{emp}(\theta) = \frac{1}{N}\sum_{n=1}^{N} L(y^{(n)}, f(x^{(n)};\theta))$$



优化方法：梯度下降

以优化模型经验风险为例，模型进行梯度下降的过程为：

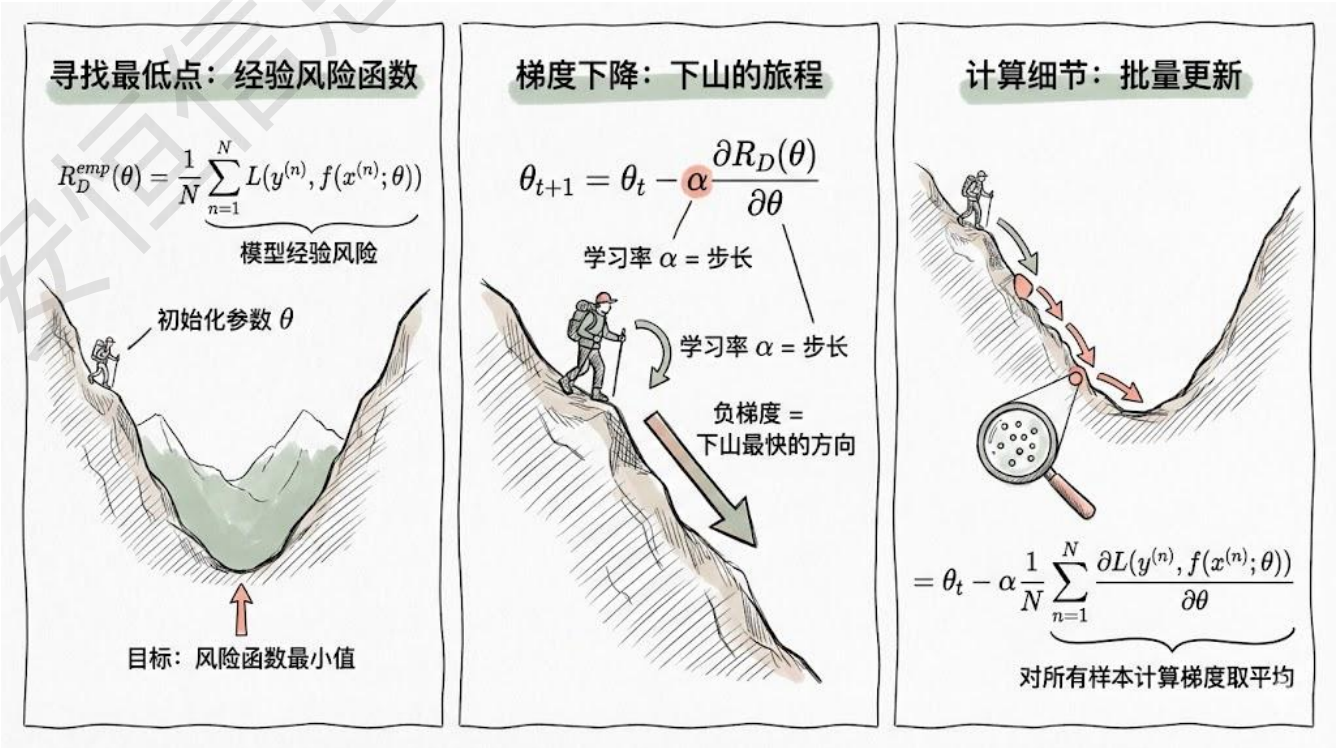
首先初始化参数θ，然后通过如下迭代公式来计算训练集上风险函数的最小值：

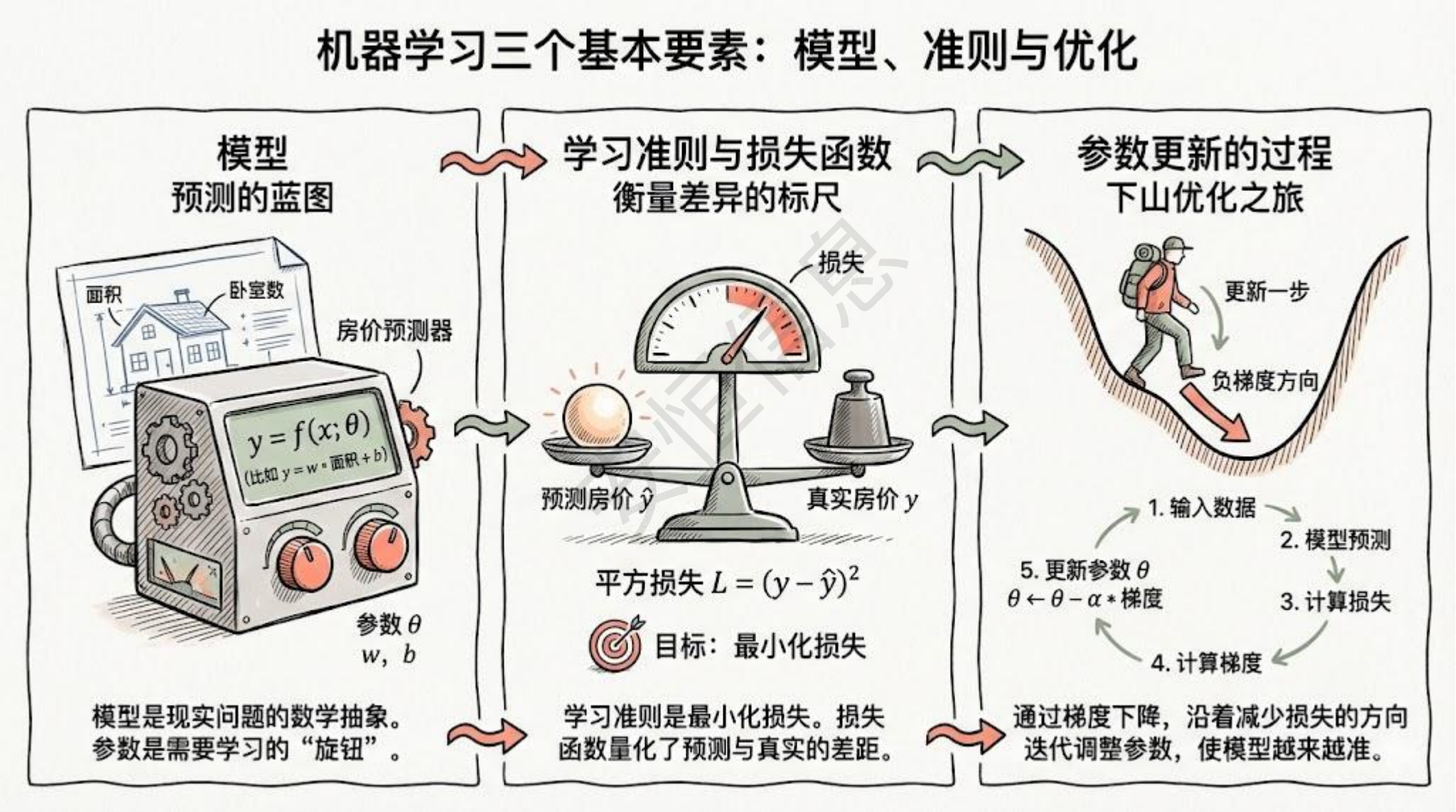
$$\theta_{t+1} = \theta_t - \alpha \frac{\partial R_D(\theta)}{\partial \theta}$$

学习率　　梯度

学习率 对应 **步长**

负梯度 对应 **下山最快的方向**




机器学习三个基本要素：模型、准则与优化
模型
预测的蓝图
面积
卧室数
房价预测器
$y = f(x;\theta)$
(比如 $y = w \cdot$ 面积 $+ b$)
参数 θ
w，b
模型是现实问题的数学抽象。
参数是需要学习的“旋钮”。
学习准则与损失函数
衡量差异的标尺
损失
预测房价 $\hat{y}$
真实房价 y
平方损失 $L = (y - \hat{y})^2$
目标：最小化损失
学习准则是最小化损失。损失
函数量化了预测与真实的差距。
参数更新的过程
下山优化之旅
更新一步
负梯度方向
1. 输入数据
2. 模型预测
3. 计算损失
4. 计算梯度
5. 更新参数 θ
$\theta \leftarrow \theta - \alpha * 梯度$
通过梯度下降，沿着减少损失的方向
迭代调整参数，使模型越来越准。

低代码模型训练案例



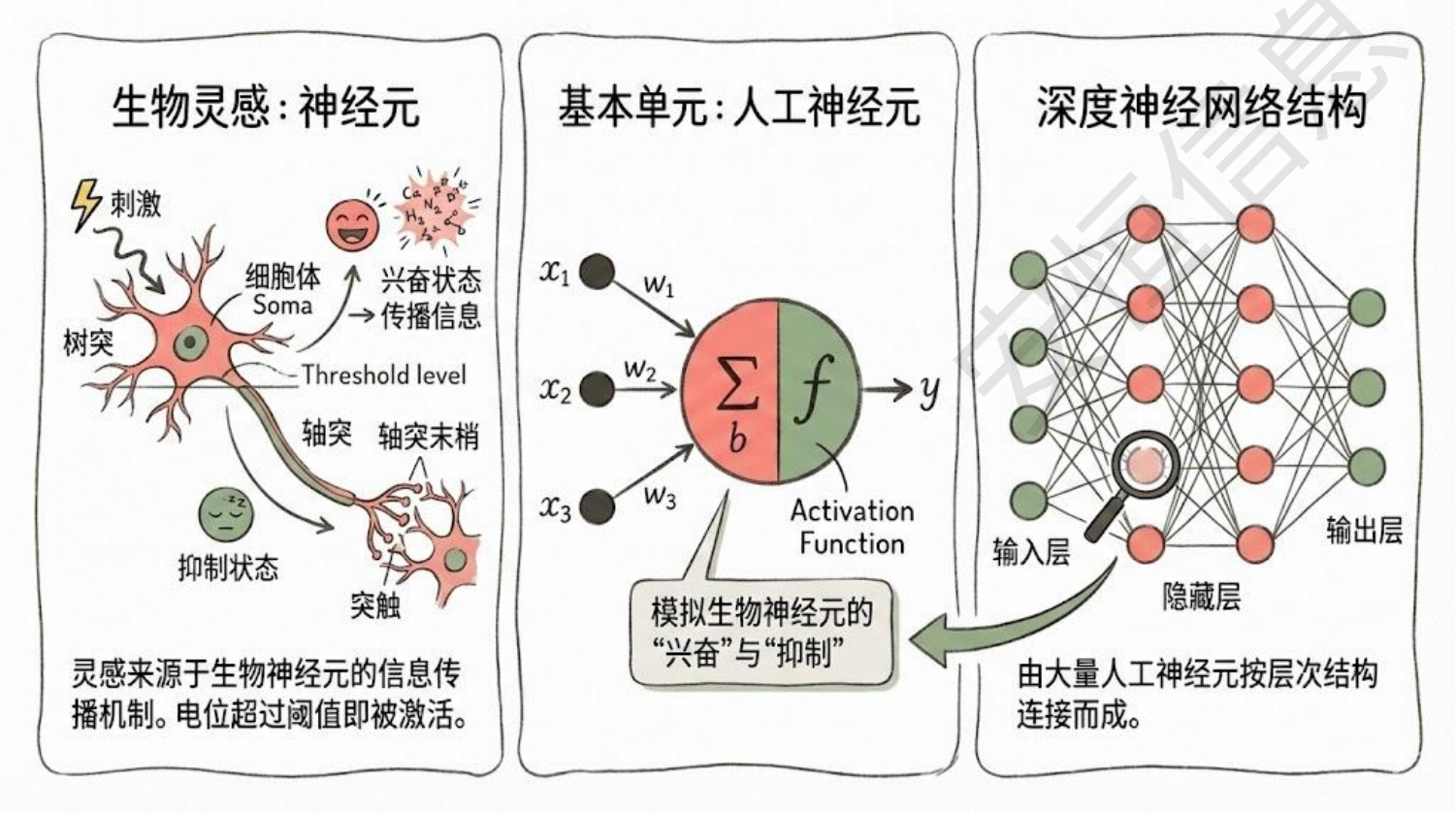
https://teachablemachine.withgoogle.com/train/image

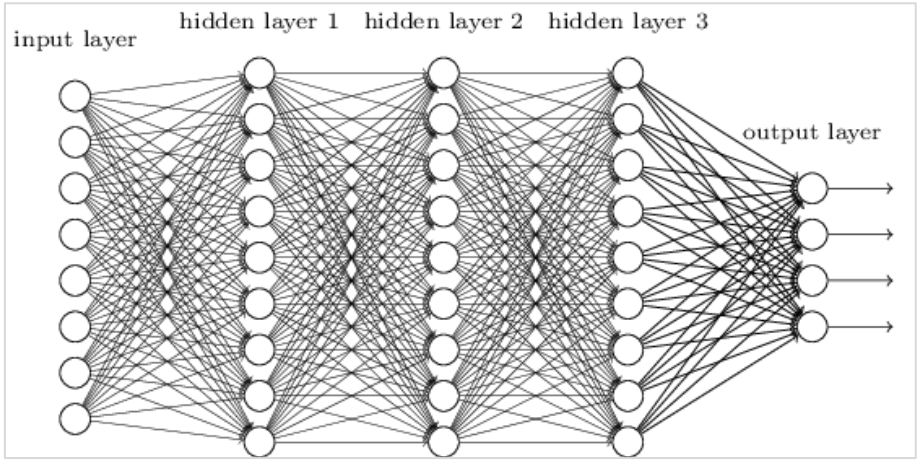
Part 02

深度神经网络

神经网络介绍

神经元是神经网络中最基本的结构，也可以说是神经网络的基本单元，它的设计灵感完全来源于生物学上神经元的信息传播机制。我们学过生物的同学都知道，神经元有两种状态：兴奋和抑制。一般情况下，大多数的神经元是处于抑制状态，但是一旦某个神经元收到刺激，导致它的电位超过一个阈值，那么这个神经元就会被 激活，处于“兴奋”状态，进而向其他的神经元传播化学物质（其实就是信息）。

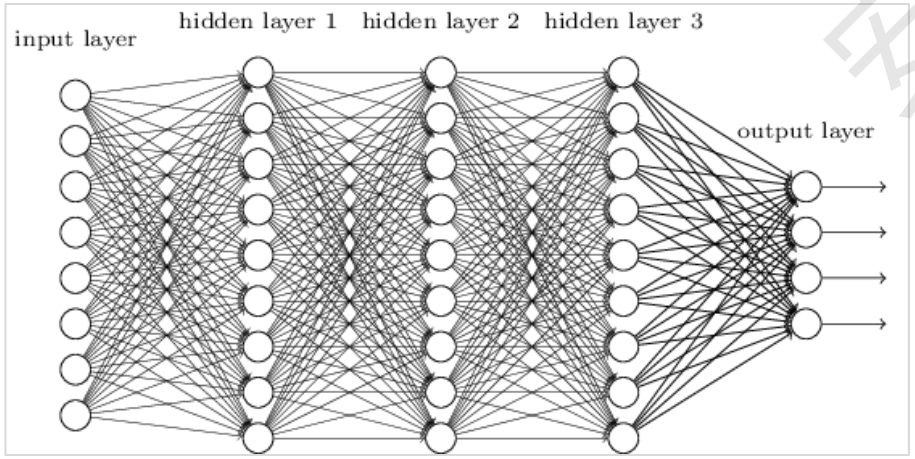




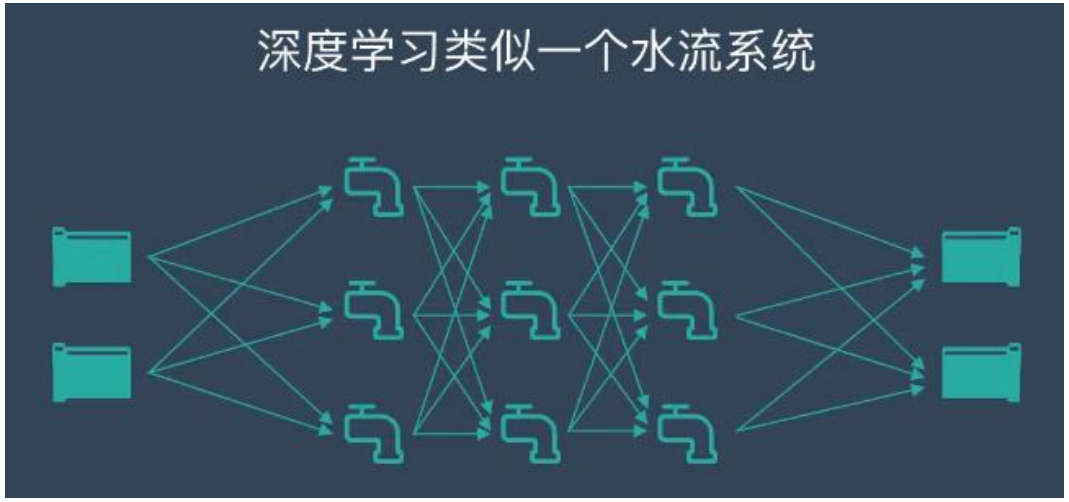
深度学习 - 神经网络结构示意图

神经网络介绍

假设深度学习要处理的信息是“水流”，而处理数据的深度神经网络是一个由管道和阀门组成的巨大水管网络。网络的入口是若干管道开口，网络的出口也是若干管道开口。这个水管网络有许多层，每一层由许多个可以控制水流流向与流量的调节阀。根据不同任务的需要，水管网络的层数、每层的调节阀数量可以有不同的变化组合。对复杂任务来说，调节阀的总数可以成千上万甚至更多。水管网络中，每一层的每个调节阀都通过水管与下一层的所有调节阀连接起来，组成一个从前到后，逐层完全连通的水流系统。（这里说的是一种比较基本的情况，不同的深度学习模型，在水管的安装和连接方式上，是有差别的）



深度学习 - 神经网络结构示意图

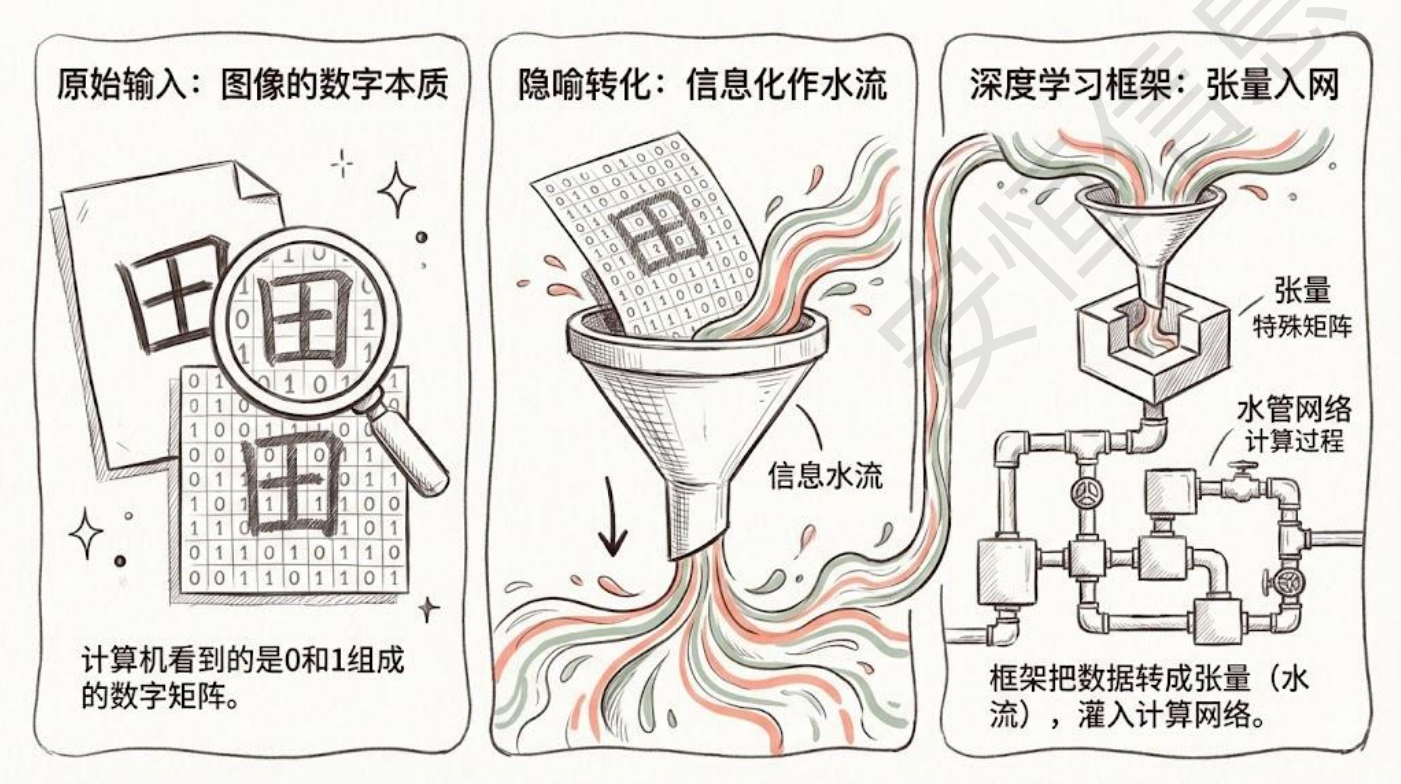


深度学习 -> 水管网络示意图

安恒信息

以图像文字识别为例，深度学习的计算过程（水管网络的排水过程）如下：

比如，当计算机看到一张写有“田”字的图片，就简单将组成这张图片的所有数字（在计算机里，图片的每个颜色点都是用“0”和“1”组成的数字来表示的）全都变成信息的水流，从入口灌进水管网络。



把图片数字化

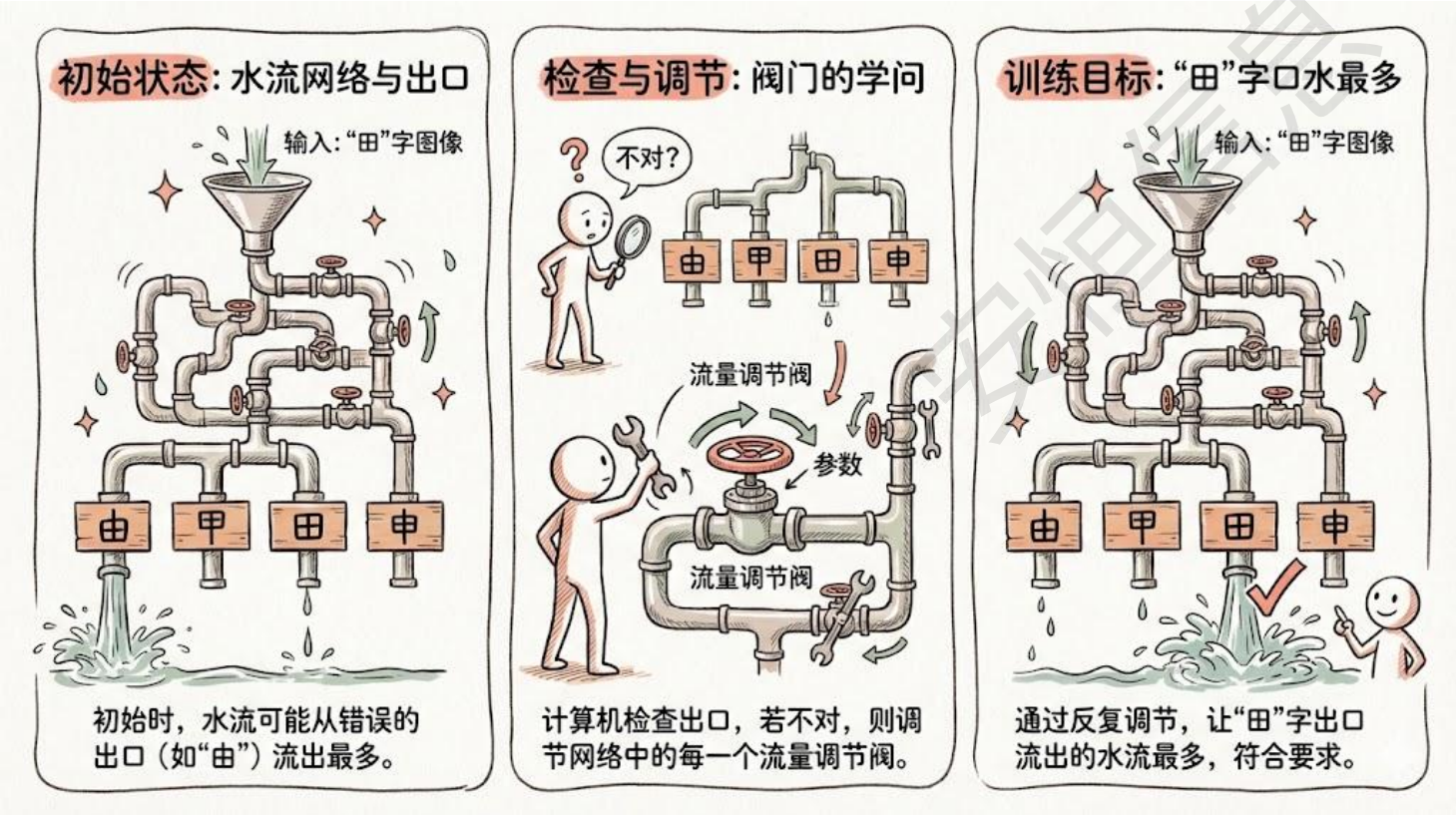


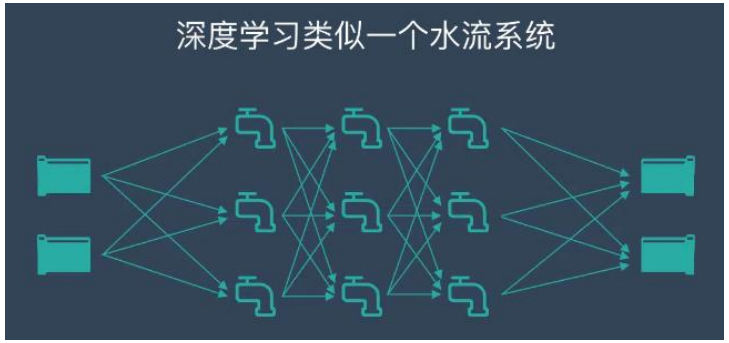
深度学习框架中：把数据从字符串转成张量（可以理解为特殊的矩阵）的形式

神经网络介绍

以图像文字识别为例，深度学习的计算（训练）过程（水管网络的排水过程）如下：

我们预先在水管网络的每个出口都插一块字牌，对应于每一个我们想让计算机认识的汉字。这时，因为输入的是“田”这个汉字，等水流流过整个水管网络，计算机就会跑到管道出口位置去看一看，是不是标记由“田”字的管道出口流出来的水流最多。如果是这样，就说明这个管道网络符合要求。如果不是这样，就调节水管网络里的每一个流量调节阀，让“田”字出口“流出”的水最多。 （模型训练过程）





神经网络介绍

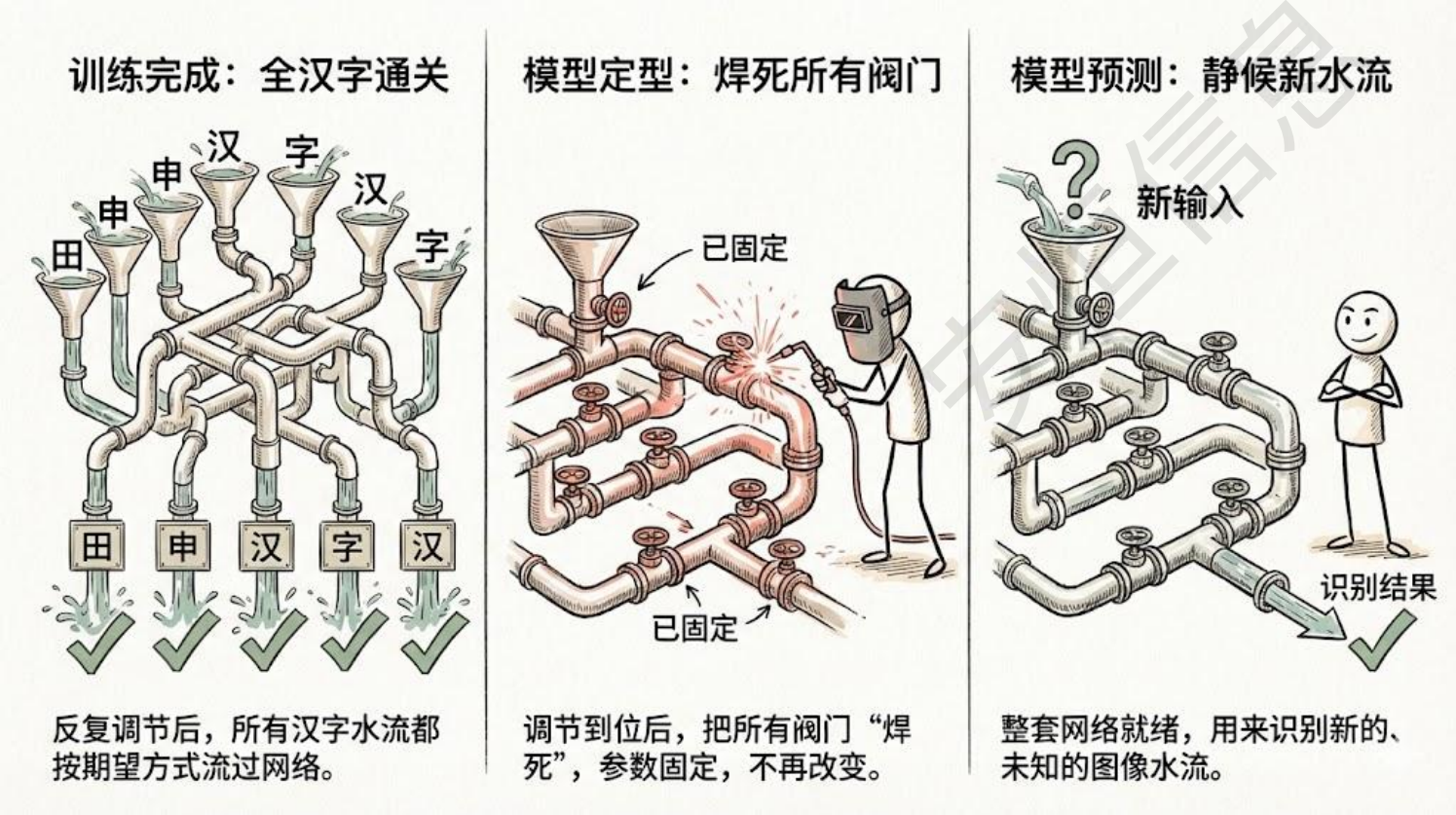
以图像文字识别为例，深度学习的计算（训练）过程（水管网络的排水过程）如下：

下一步，学习“申”字时，我们就用类似的方法，把每一张写有“申”字的图片变成一大堆数字组成的水流，灌进水管网络，看一看，是不是写有“申”字的那个管道出口流水最多，如果不是，我们还得再调整所有的阀门。这一次，要既保证刚才学过的“田”字不受影响，也要保证新的“申”字可以被正确处理。 （模型训练过程）



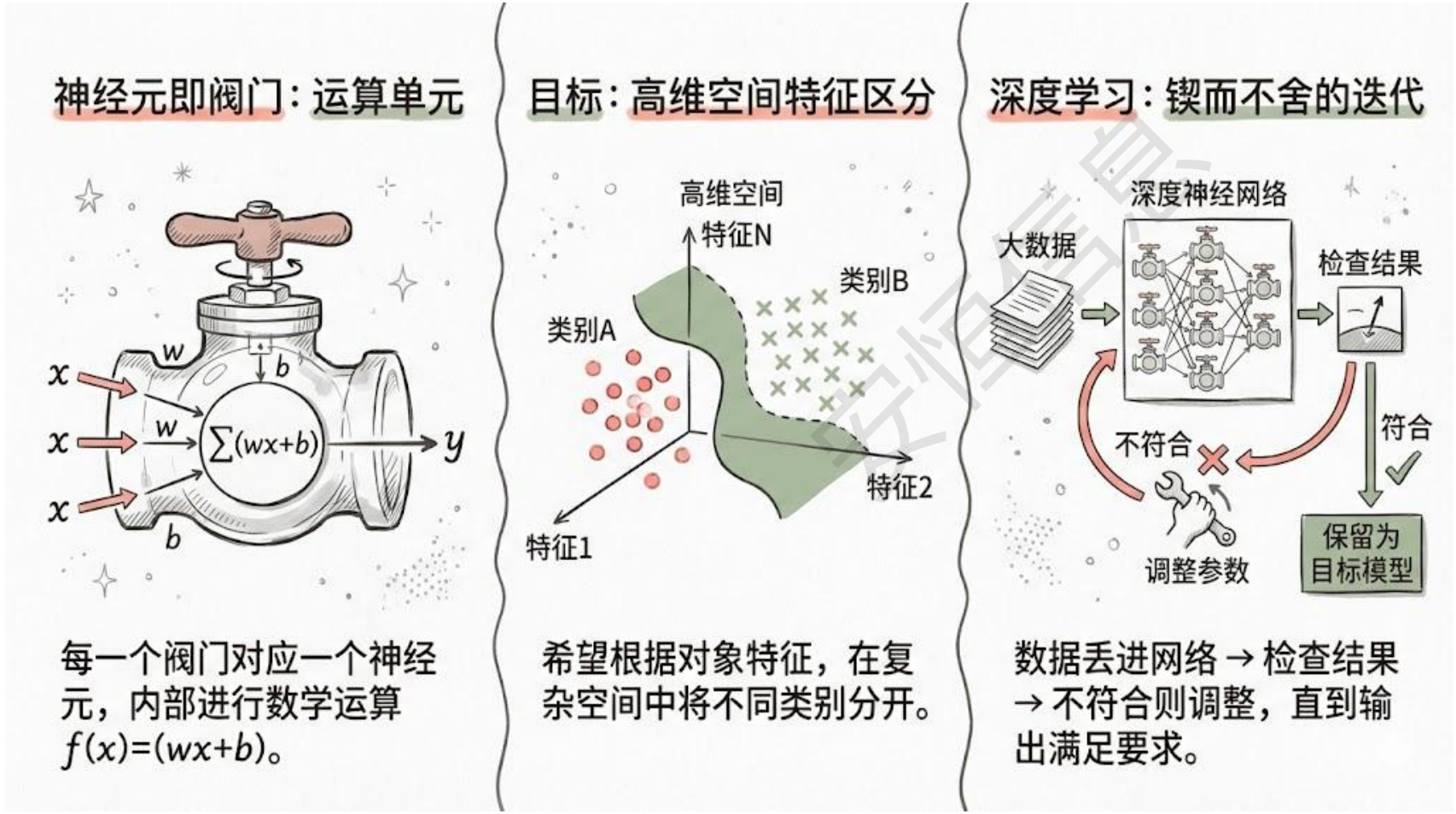
以图像文字识别为例，深度学习的计算（推理）过程（水管网络的排水过程）如下：

如此反复进行，知道所有汉字对应的水流都可以按照期望的方式流过整个水管网络。这时，我们就说，这个水管网络是一个训练好的深度学习模型了。当大量汉字被这个管道网络处理，所有阀门都调节到位后，整套水管网络就可以用来识别汉字了。这时，我们可以把调节好的所有阀门都“焊死”，静候新的水流到来。（模型预测过程）



神经网络介绍

对于深度学习来讲，每一个阀门对应一个神经元，每一个神经元都会在内部进行数学运算，希望在高维空间中，根据对象特征，将不同类别的对象区分开来。



DNN 基本结构

神经网络架构的灵感源自人脑。人脑细胞称为神经元，构成了一个复杂、高度互联的网络，并能互相发送电信号，以帮助人类处理信息。同样，人工神经网络由人工神经元组成，它们共同合作以解决问题。人工神经元是软件模块，称为节点；人工神经网络是软件程序或算法，在其核心，使用计算系统求解数学计算。

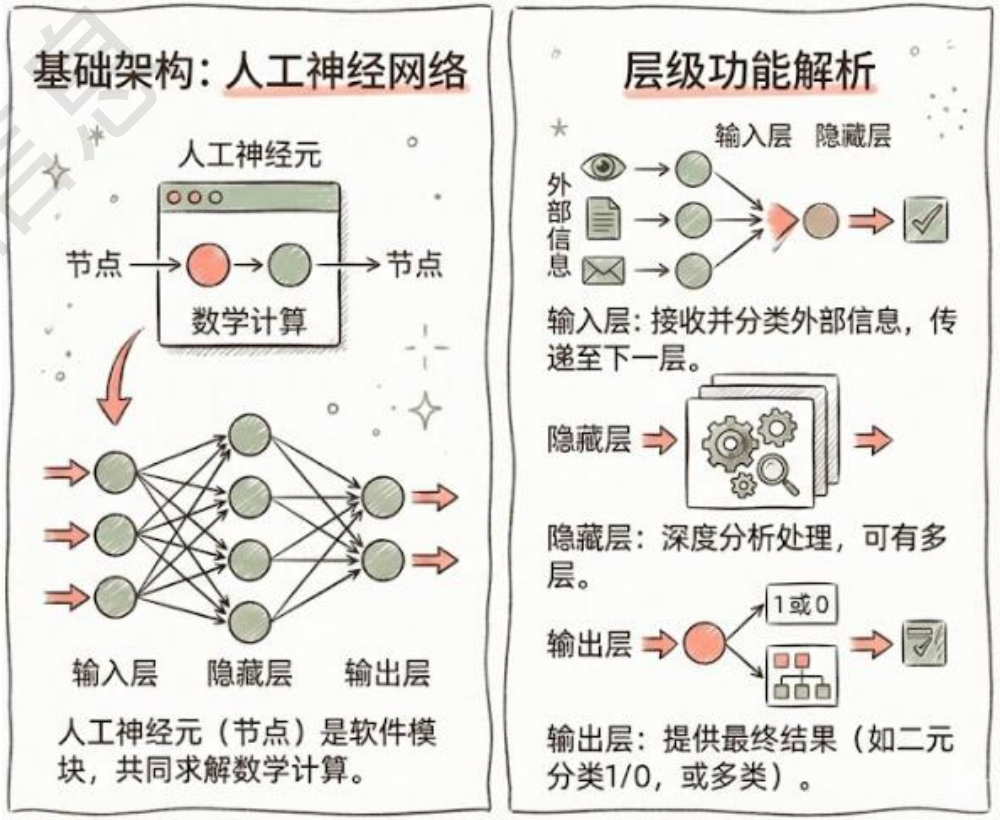
简单神经网络架构：

基本神经网络的相互连接的人工神经元分为三层：

- **输入层：**来自外部世界的信息通过输入层进入人工神经网络。输入节点对数据进行处理、分析或分类，然后将其继续传递到下一层。

- **隐藏层：**隐藏层从输入层或其他隐藏层获取其输入。人工神经网络可以具有大量的隐藏层。每个隐藏层都会对来自上一层的输出进行分析和进一步处理，然后将其继续传递到下一层。

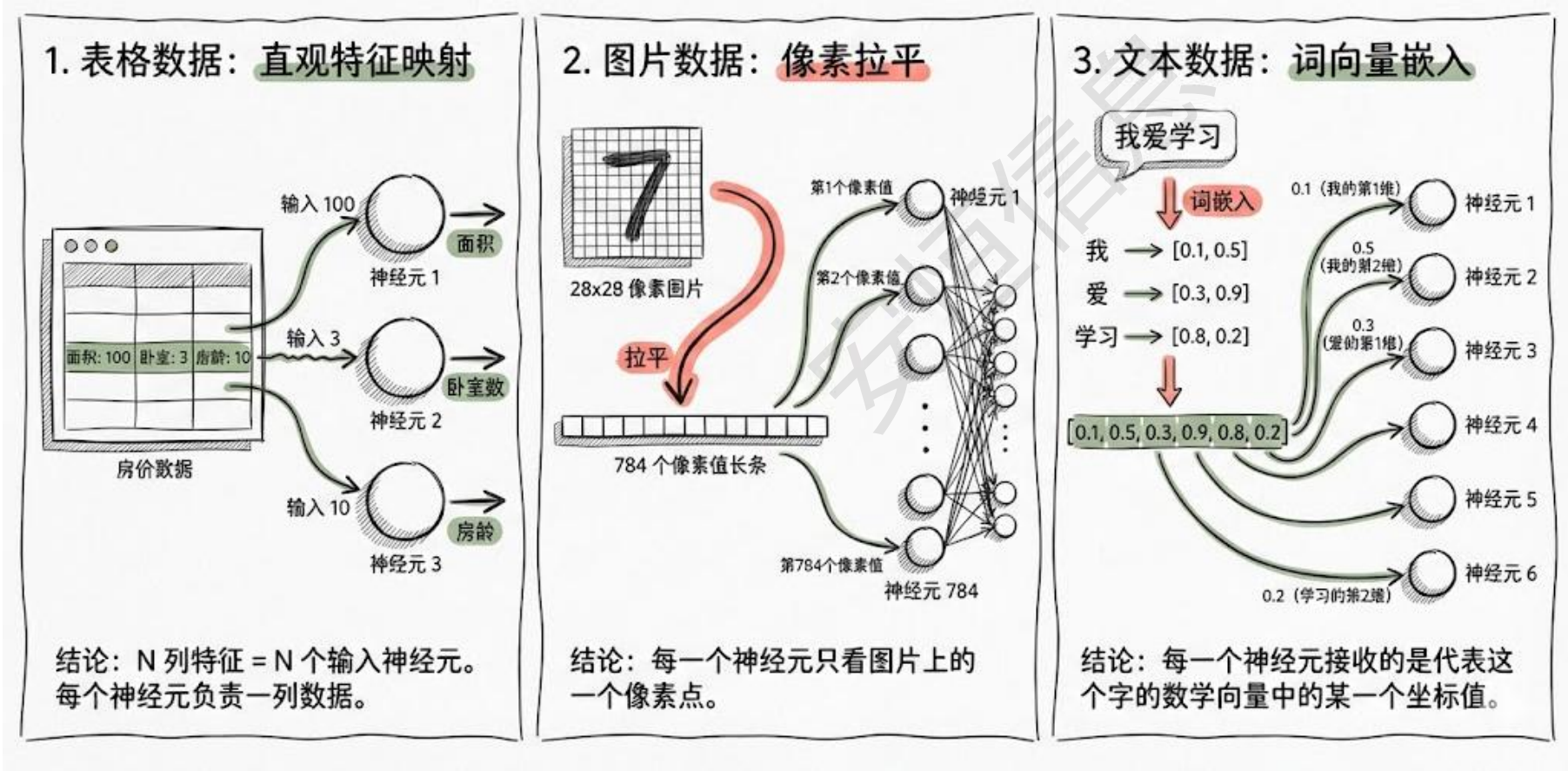
- **输出层：**输出层提供人工神经网络对所有数据进行处理的最终结果。它可以包含单个或多个节点。例如，如果我们要解决一个二元（是/否）分类问题，则输出层包含一个输出节点，它将提供 1 或 0 的结果。但是，如果我们要解决一个多类分类问题，则输出层可能会由一个以上输出节点组成。



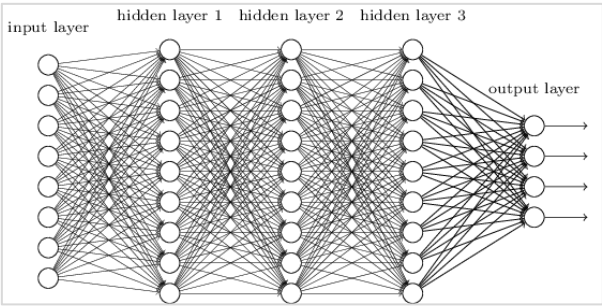
安恒信息

DNN 基本结构

DNN（深度神经网络）按不同层的位置划分内部的神经网络层可以分为三类，**输入层**，**隐藏层**和**输出层**。如下图示例，一般来说第一层是输入层，最后一层是输出层，而中间的层数都是隐藏层。



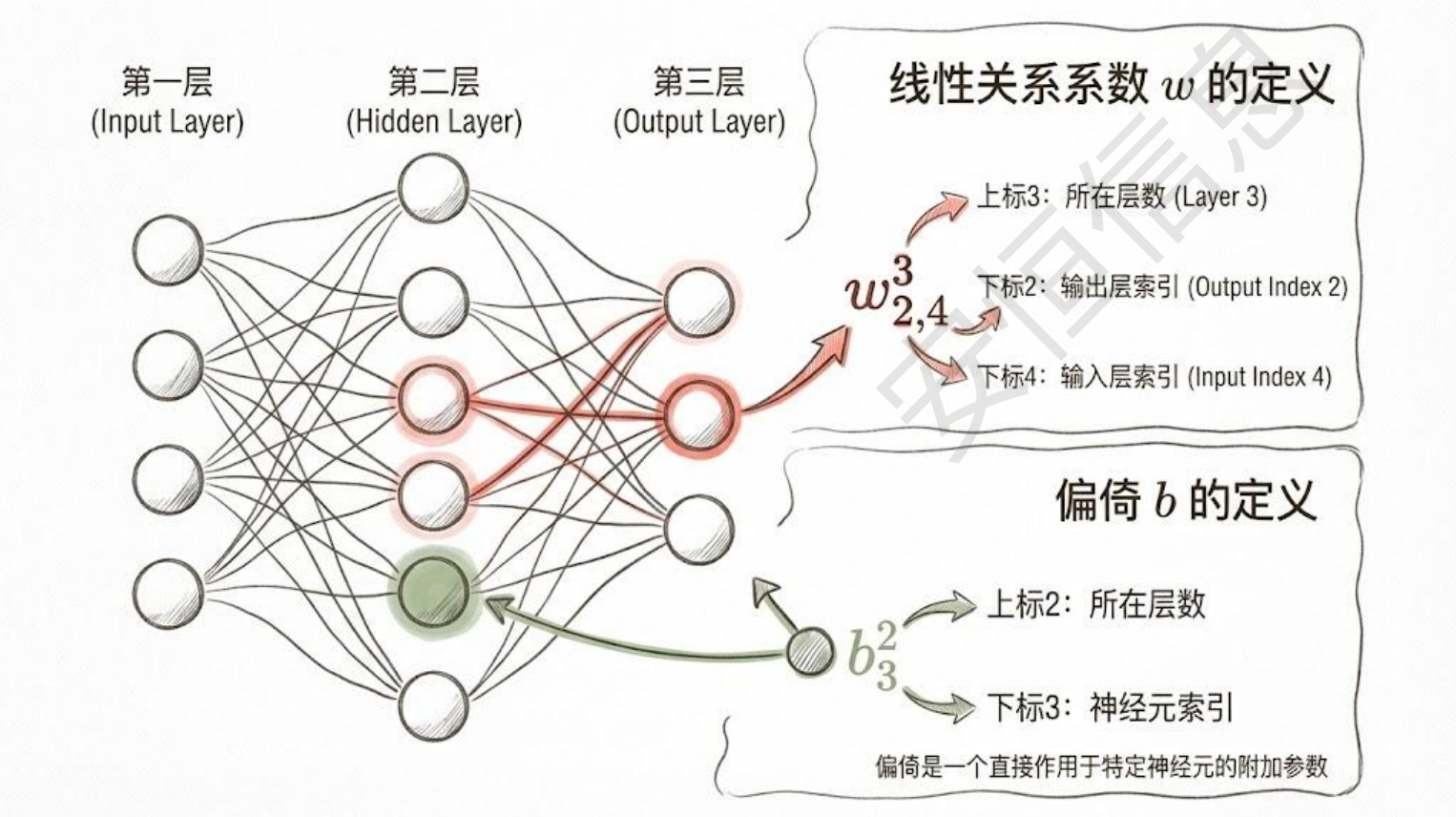
其中，层与层之间是**全连接**的，也就是说，第 i 层的任意一个神经元一定与第 i+1 层的任意一个神经元相连。



DNN 基本结构

首先我们来看看线性关系系数 w 和偏倚 b 的定义。以下图一个三层的 DNN 为例，第二层的第 4 个神经元到第三层的第 2 个神经元 的线性系数定义为 $w^3_{2,4}$,上标3 代表 线性系数 w 所在的层数，而 下标 对应的是 输出的第三层索引 2 和输入的第二层索引 4 。

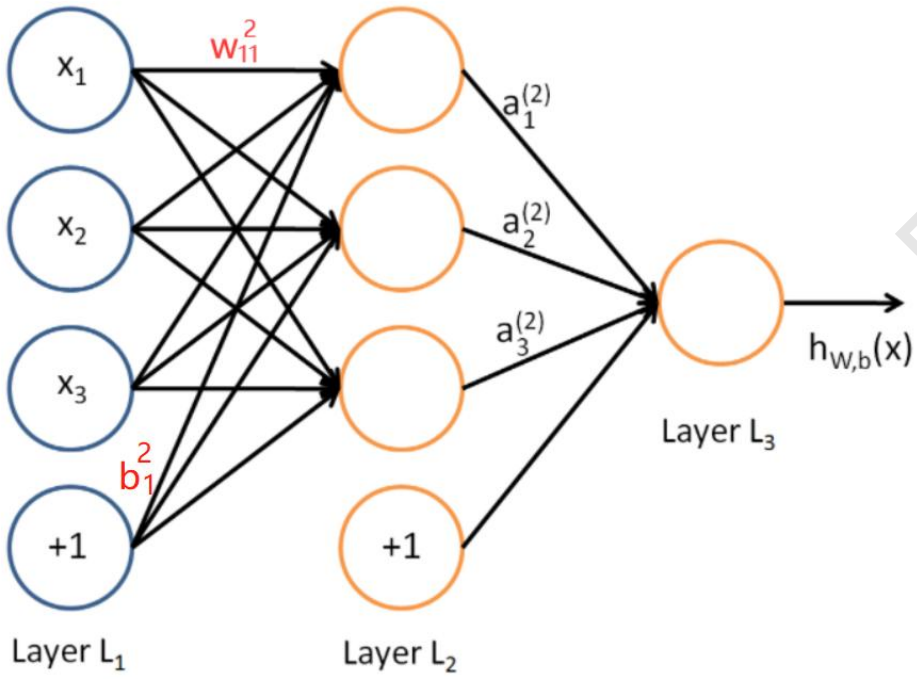
第二层的第三个神经元对应的偏倚定义为 b^2_3 。其中，上标 2 代表所在的层数，下标 3 代表偏倚所在的神经元的索引。



DNN 前向传播

所谓的 **DNN（深度神经网络）** 的 **前向传播算法** 也就是利用我们的若干个 **权重系数矩阵 W，偏倚向量 b** 来和**输入值向量 x** 进行一系列 **线性运算** 和 **激活运算**，从输入层开始，一层层的向后计算，一直到运算到输出层，得到输出结果为值。

假设我们选择的激活函数是 σ(z)，隐藏层和输出层的输出值为 a，则对于下图的三层 DNN ,利用和感知机一样的思路，我们可以利用上一层的输出计算下一层的输出，则有：



对于 **第二层** 的输出 $a_1^2 \quad a_2^2 \quad a_3^2$，我们有：

$$a_1^2 = \sigma(z_1^2) = \sigma(w_{11}^2 x_1 + w_{12}^2 x_2 + w_{13}^2 x_3 + b_1^2)$$

$$a_2^2 = \sigma(z_2^2) = \sigma(w_{21}^2 x_1 + w_{22}^2 x_2 + w_{23}^2 x_3 + b_2^2)$$

$$a_3^2 = \sigma(z_3^2) = \sigma(w_{31}^2 x_1 + w_{32}^2 x_2 + w_{33}^2 x_3 + b_3^2)$$

对于 **第三层** 的输出 a_1^3 我们有：

$$a_1^3 = \sigma(z_1^3) = \sigma(w_{11}^3 a_1^2 + w_{12}^3 a_2^2 + w_{13}^3 a_3^2 + b_1^3)$$

安恒信息

将上例的式子一般化，假设第 l-1 层共有 m 个神经元，则对于第 l 层的第 j 个神经元的输出 a_j^l ，我们有：

$$a_1^3 = \sigma(z_1^3) = \sigma(w_{11}^3 a_1^2 + w_{12}^3 a_2^2 + w_{13}^3 a_3^2 + b_1^3)$$

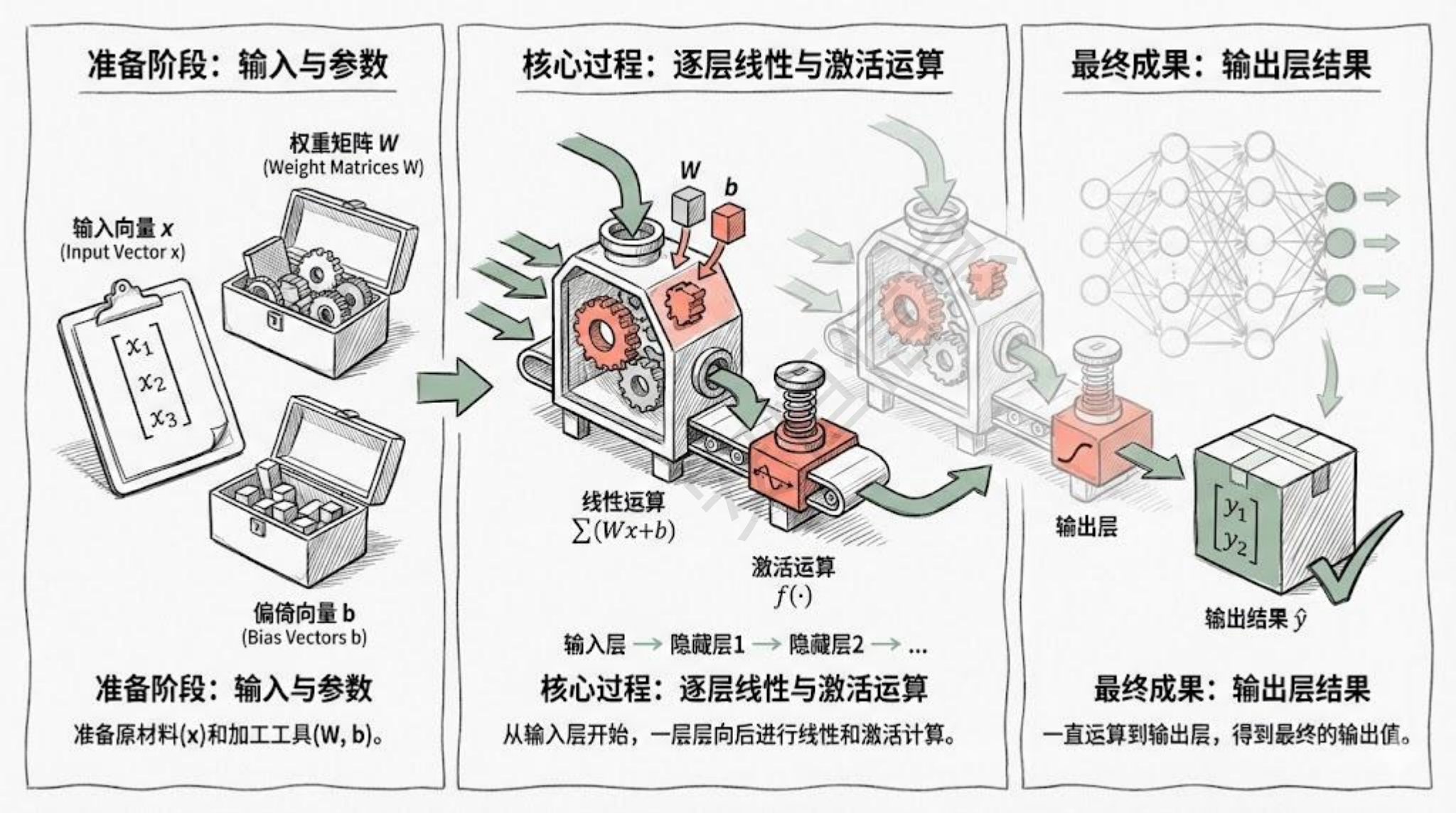
$$a_j^l = \sigma(z_j^l) = \sigma(\sum_{k=1}^{m} w_{jk}^l a_k^{l-1} + b_j^l)$$

如果将其转化为矩阵表示：

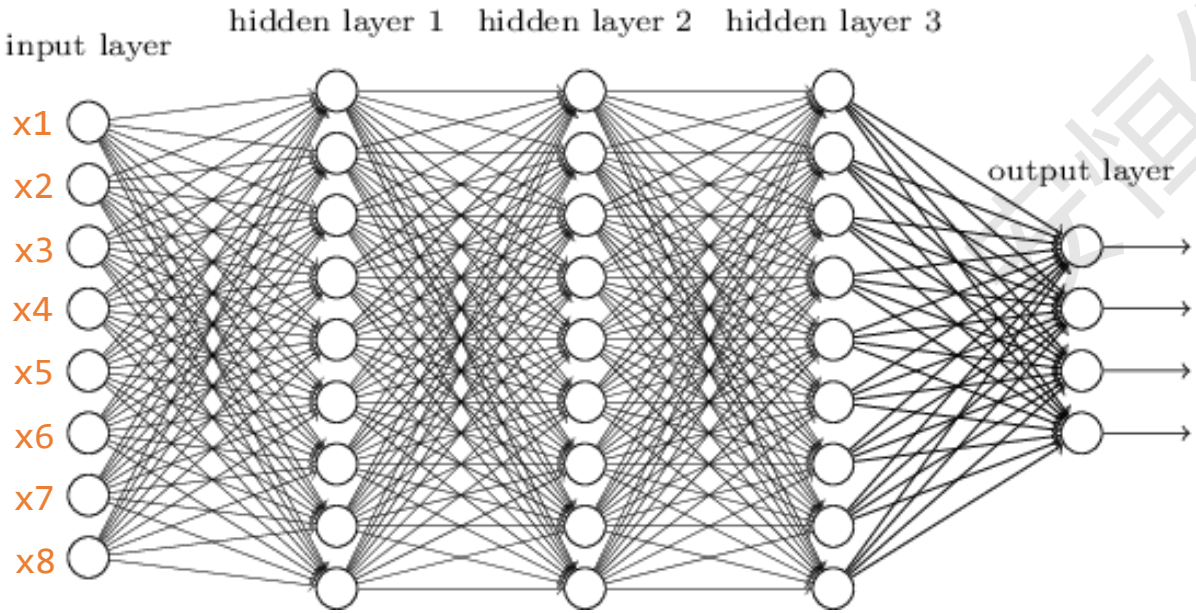
$$a^l = \sigma(z^l) = \sigma(W^l a^{l-1} + b^l)$$

安恒信息

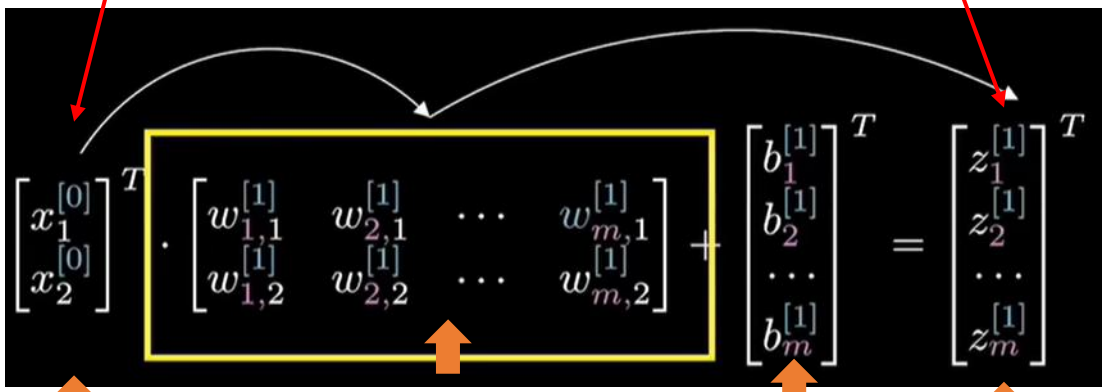
DNN 前向传播小结


准备阶段：输入与参数
权重矩阵 W
(Weight Matrices W)
输入向量 x
(Input Vector x)
x_1
x_2
x_3
偏倚向量 b
(Bias Vectors b)
核心过程：逐层线性与激活运算
W
b
线性运算
$\sum(Wx+b)$
激活运算
$f(\cdot)$
输入层 → 隐藏层1 → 隐藏层2 → ...
最终成果：输出层结果
输出层
y_1
y_2
输出结果 $\hat{y}$
准备阶段：输入与参数
准备原材料(x)和加工工具(W, b)。
核心过程：逐层线性与激活运算
从输入层开始，一层层向后进行线性和激活计算。
最终成果：输出层结果
一直运算到输出层，得到最终的输出值。

可以看到，神经网络的结构一般是先进行升维（神经元个数增多），再进行降维（神经元个数减少）的操作，这有什么意义？



对于神经网络我们可以把他看成是一个黑盒，只不过是把输入进去的数据（向量x）和一个矩阵相乘，然后得到一个新的向量（向量z）。



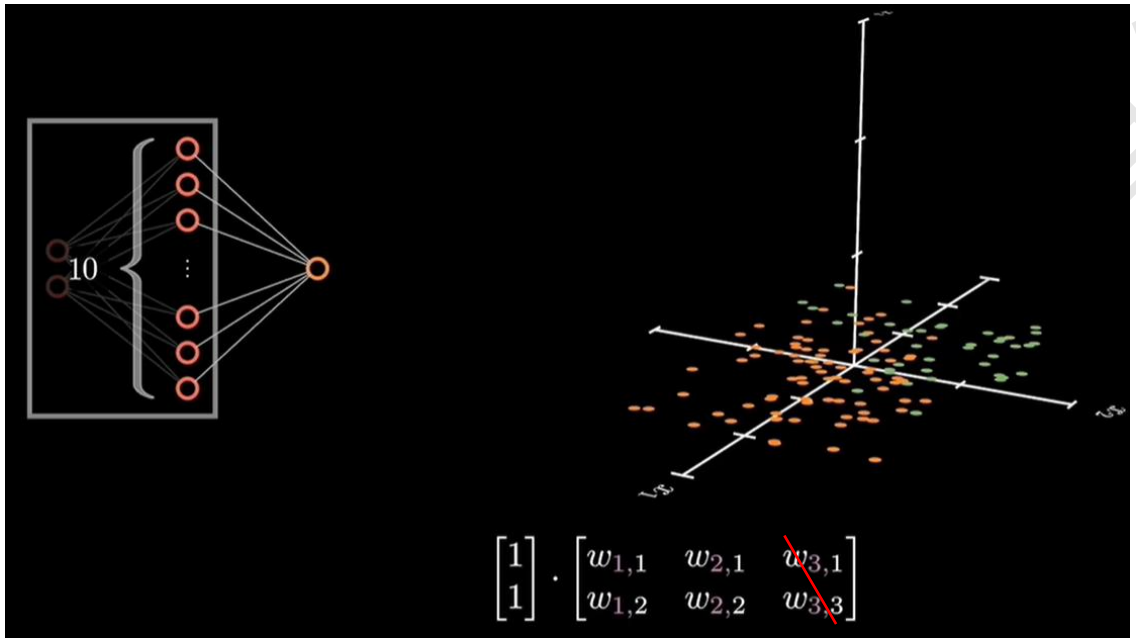
输入是两个特征的数据

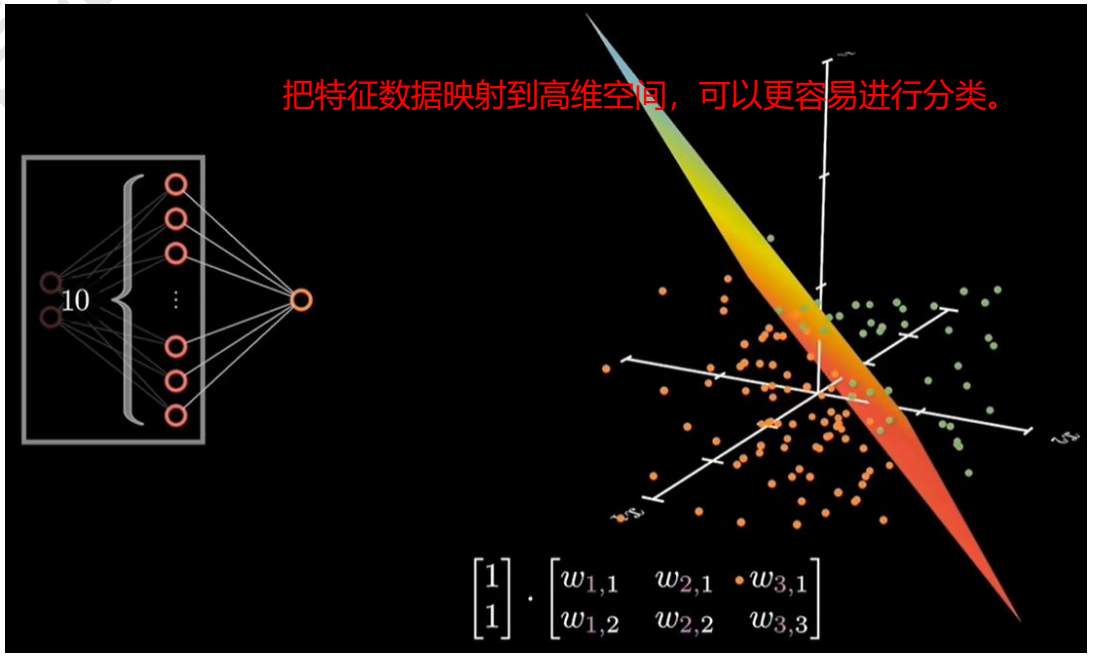
m个神经元

偏置项

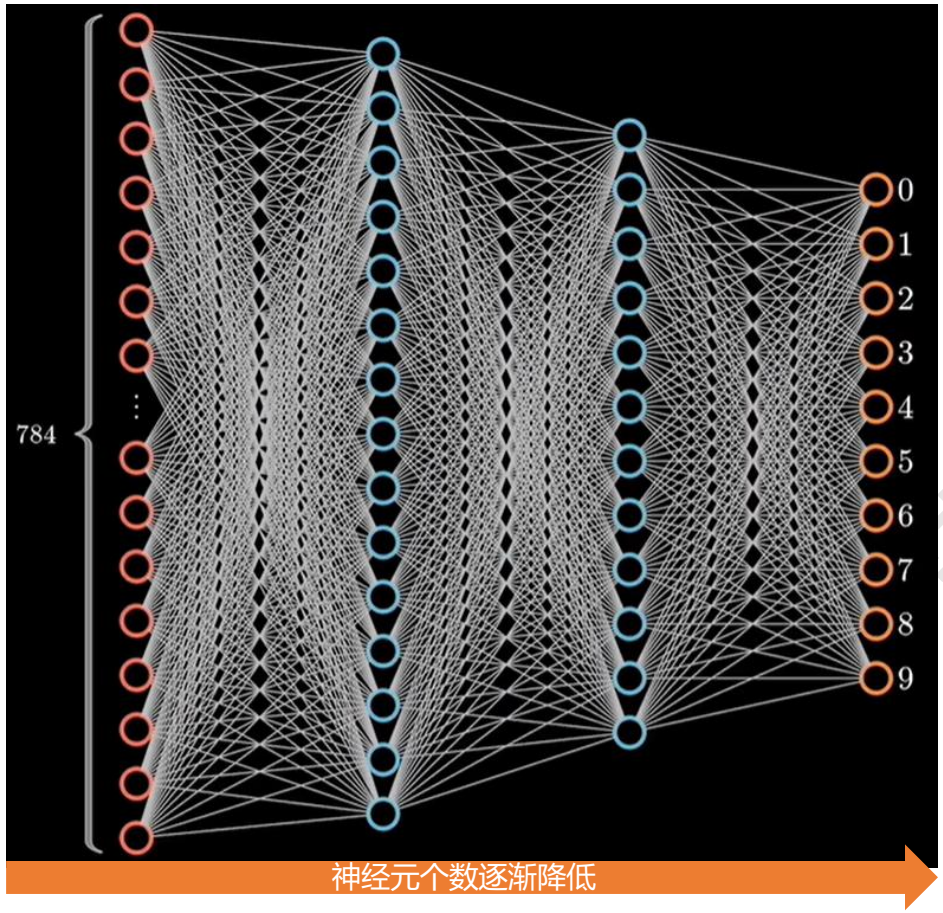
新的向量（可以看到升维了）

隐藏层神经元个数的变化 → 升维：

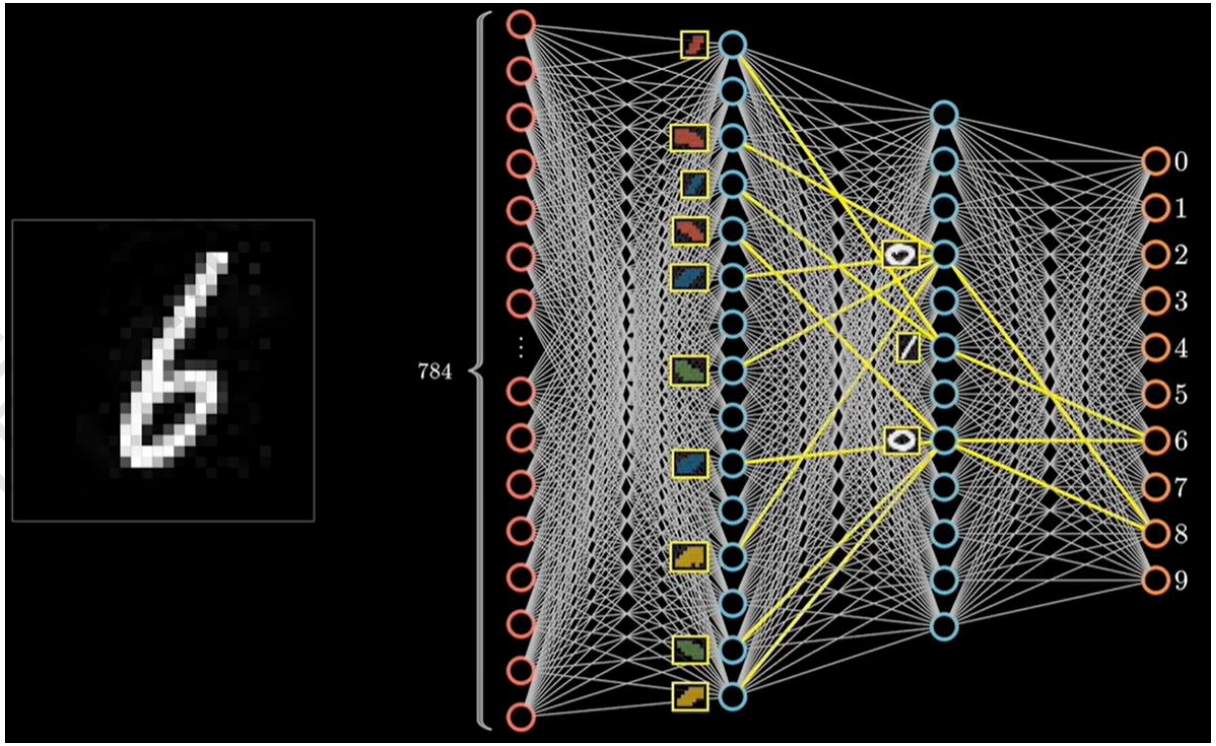




隐藏层神经元个数的变化 → 降维：







因为数据的特征是可复用的，所以在最终进行预测的时候，神经元个数可以减少

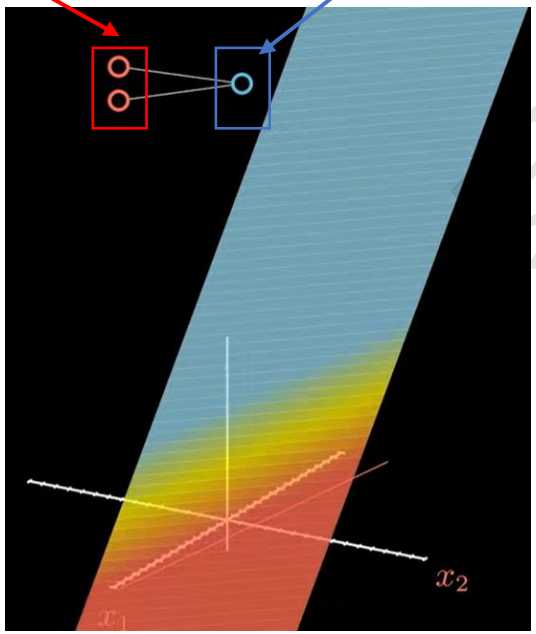
DNN 前向传播

激活函数的作用：

二维参数（x1,x2），经过一个神经元计算，,最终得到 z

$$\begin{bmatrix} x_1 \\ x_2 \end{bmatrix}^T \cdot \begin{bmatrix} w_1 \\ w_2 \end{bmatrix} + [b] = [z]$$

$$sigmoid(z) = a$$

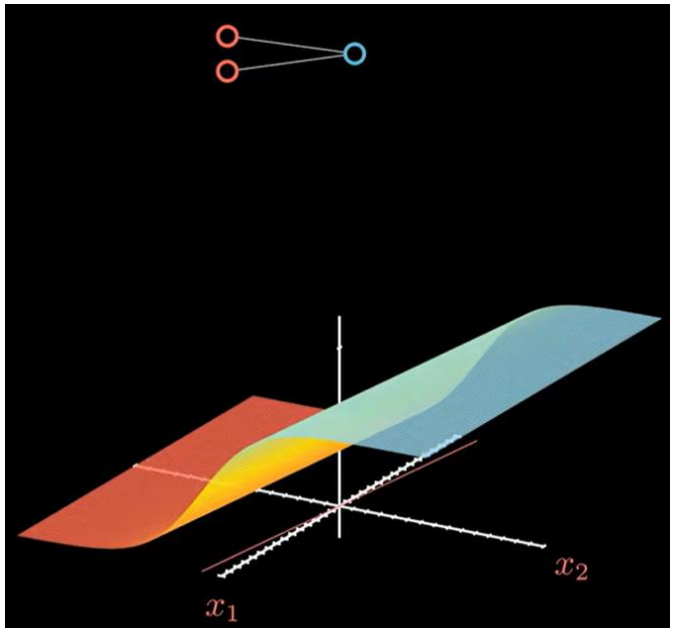


二维情况下，模型拟合出一个平面

神经元接收二维参数（x1,x2），进行计算，最终得到 z ，z 再经过激活函数激活后得到 a ，可以可能到此时的函数图像不再是一个平面，而是一个曲面。

$$\begin{bmatrix} x_1 \\ x_2 \end{bmatrix}^T \cdot \begin{bmatrix} w_1 \\ w_2 \end{bmatrix} + [b] = [z]$$

$$sigmoid(z) = a$$



二维情况下，经过激活函数，模型拟合出一个曲面

安恒信息

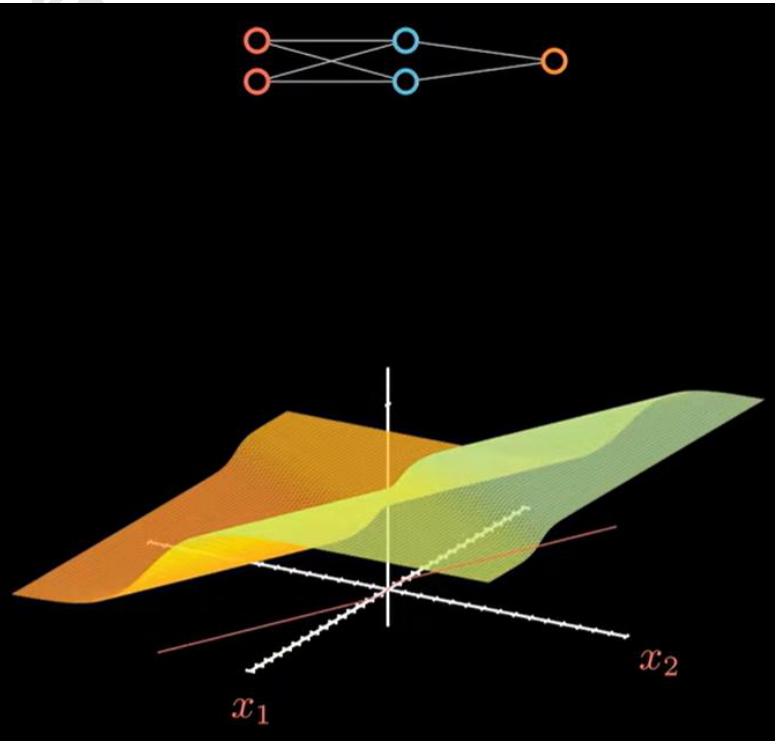
神经元个数变化对于模型表达形式带来的影响：（二维数据，2个神经元 + 激活函数 = 最终结果 ）

$$\begin{bmatrix} x_1^{[0]} \\ x_2^{[0]} \end{bmatrix}^T \cdot \begin{bmatrix} w_{1,1}^{[1]} & w_{2,1}^{[1]} \\ w_{1,2}^{[1]} & w_{2,2}^{[1]} \end{bmatrix} + \begin{bmatrix} b_1^{[1]} \\ b_2^{[1]} \end{bmatrix}^T = \begin{bmatrix} z_1^{[1]} \\ z_2^{[1]} \end{bmatrix}^T$$

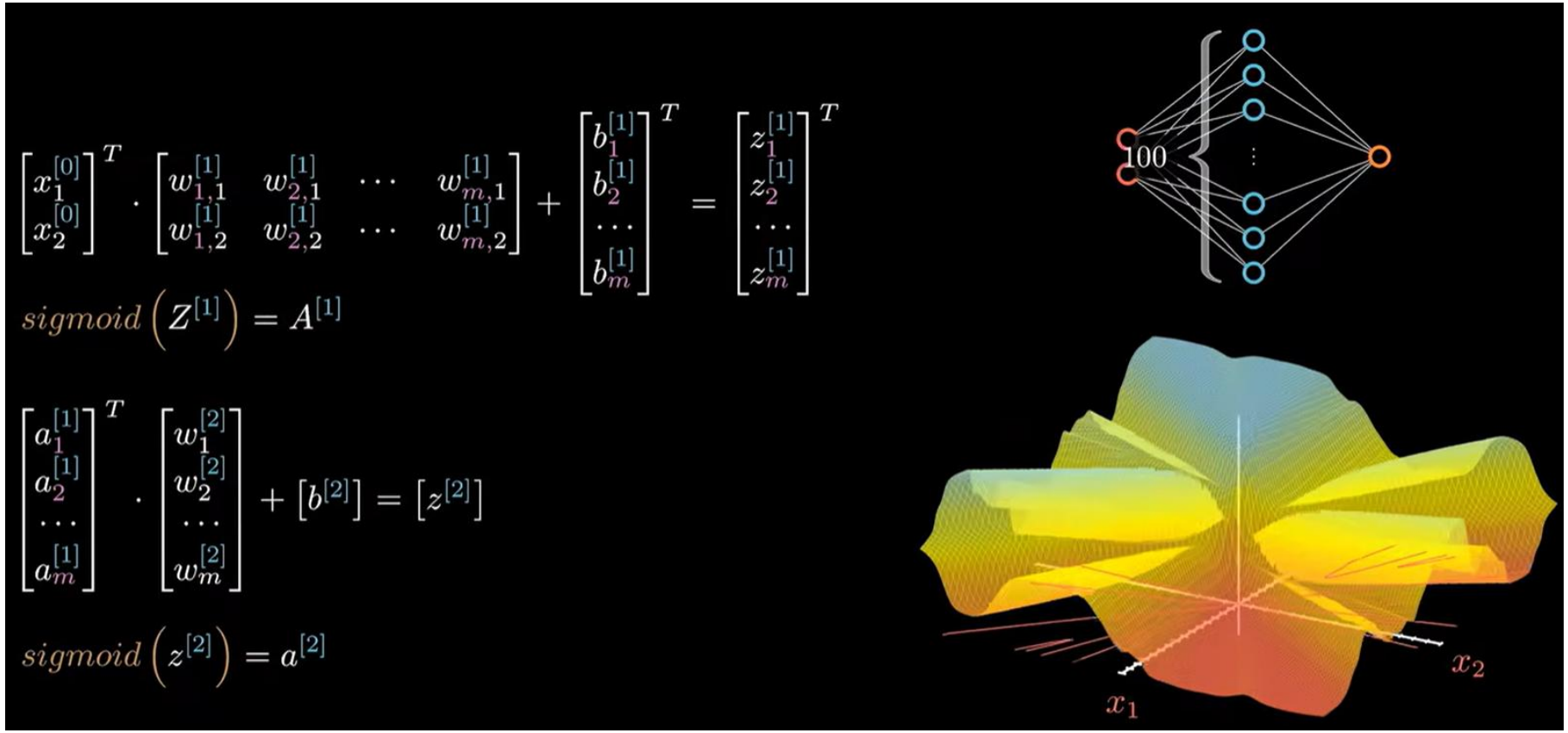
$$sigmoid\left(Z^{[1]}\right) = A^{[1]}$$

$$\begin{bmatrix} a_1^{[1]} \\ a_2^{[1]} \end{bmatrix}^T \cdot \begin{bmatrix} w_1^{[2]} \\ w_2^{[2]} \end{bmatrix} + \left[b^{[2]}\right] = \left[z^{[2]}\right]$$

$$sigmoid\left(z^{[2]}\right) = a^{[2]}$$

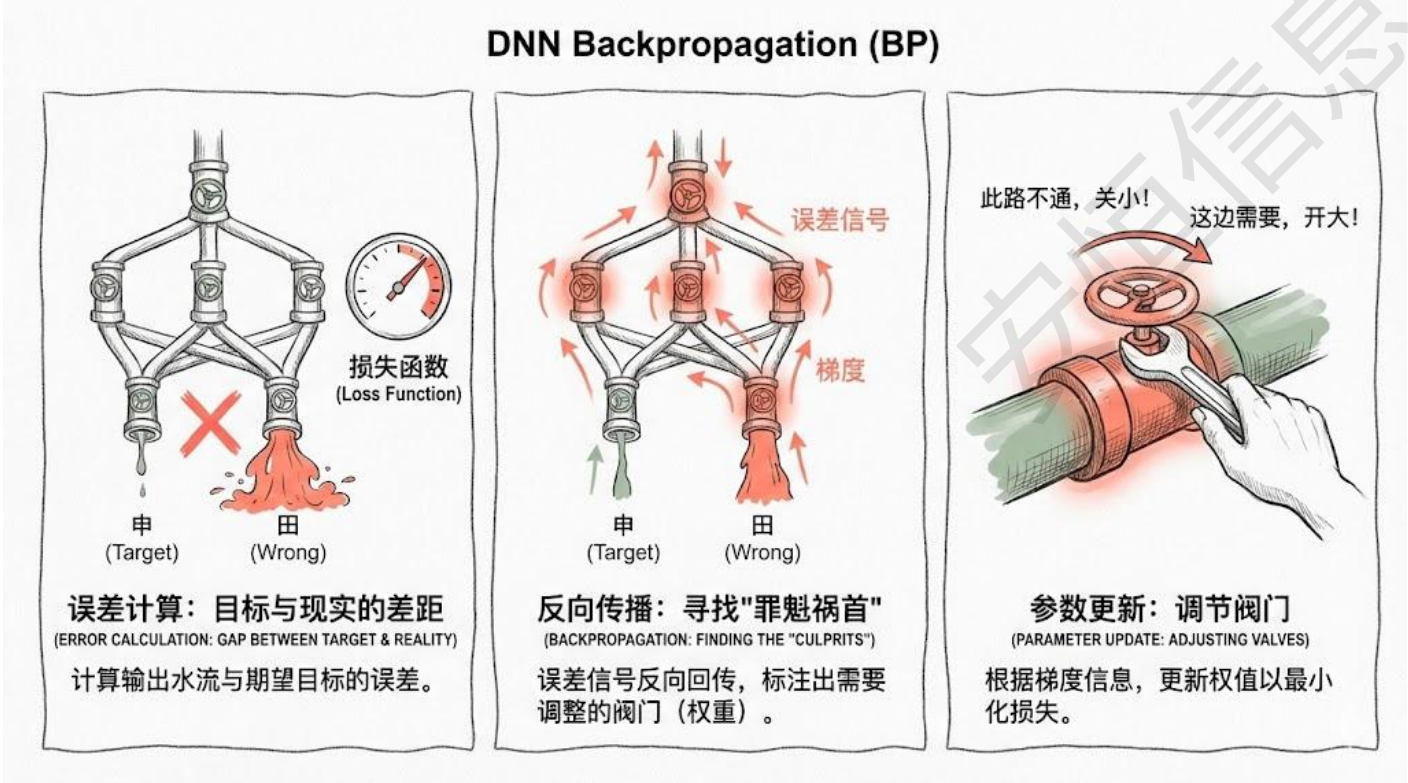


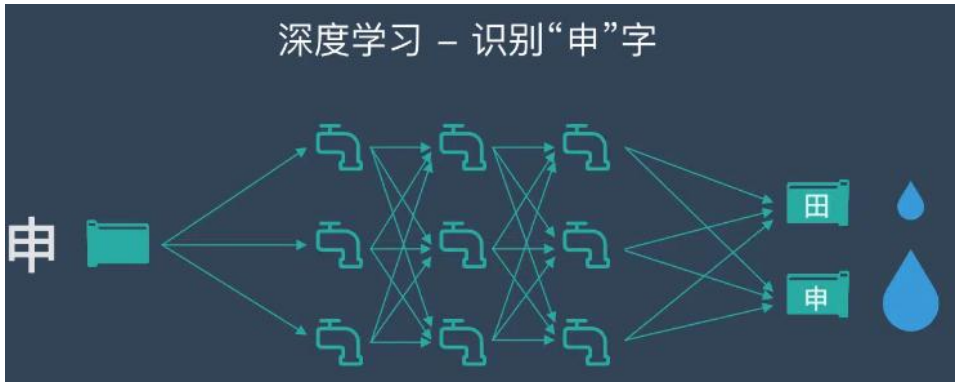
神经元个数变化对于模型表达形式带来的影响：（二维数据，100个神经元 + 激活函数 = 最终结果 ）



DNN 反向传播（BP算法）

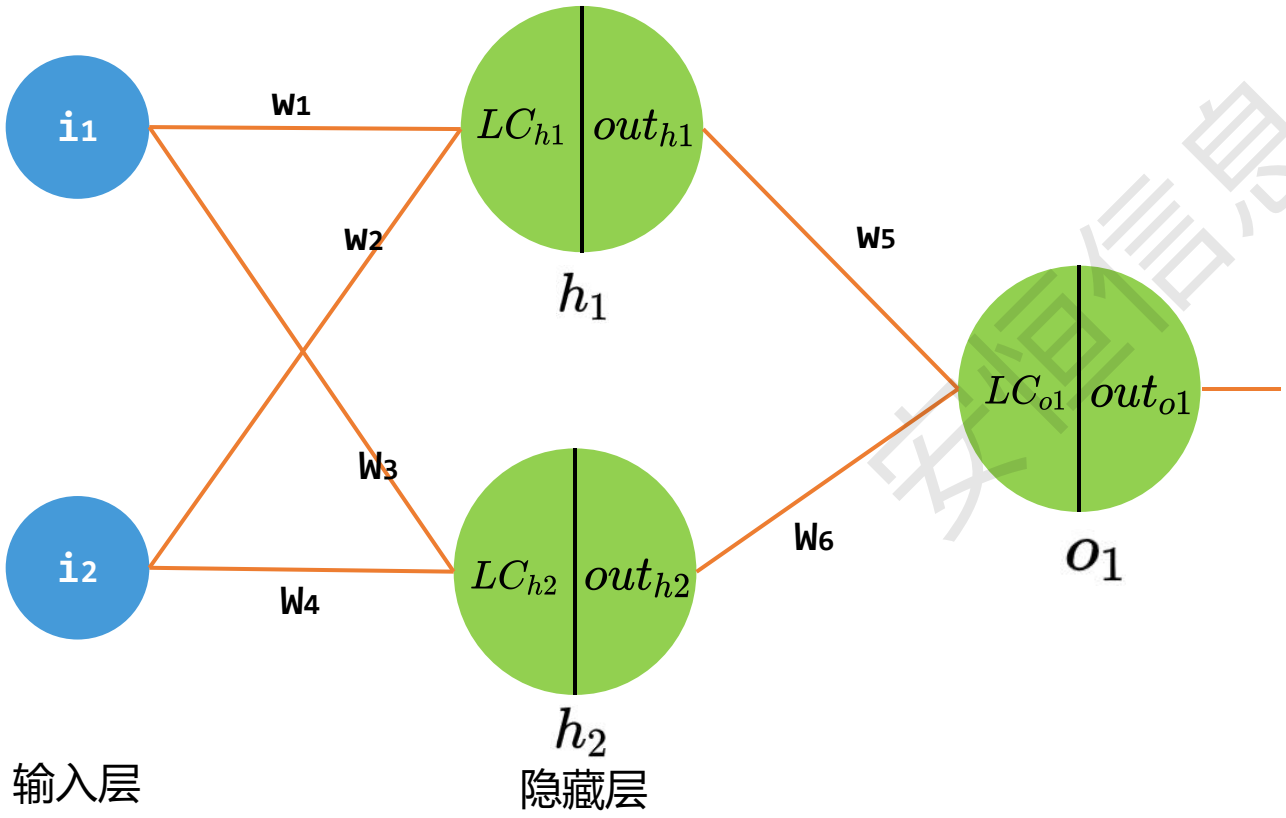
BP算法是“误差反向传播”的简称，是一种与最优化方法（如梯度下降法）结合使用的，用来训练人工神经网络的常见方法。该方法对网络中所有权重计算损失函数的梯度。这个梯度会反馈给最优化方法，用来更新权值以最小化损失函数。



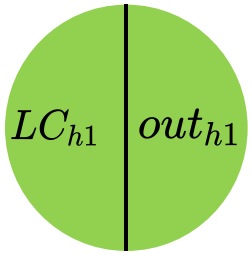


反向传播就是让模型更新参数（调节水管），可以使得我们可以尽可能靠近我们的目标（让申字牌的水管流出的水最多）。

现有三层神经网络，场景如下所示：



其中：



LC（Linear combination）意为线性组合，LC_{h1} 表示隐藏层 h_1 的输入值，也就是 i_1 输入层经过线性组合的输出值。

out_{h1} 表示隐藏层 h_1 经过激活函数转换后的输出值。

安恒信息

1）定义损失函数，采用 **均方损失** 为例：$$Loss_{total} = \frac{1}{2}\sum(outer - target)^2$$

2）定义激活函数：**采用 Sigmoid 激活函数**：$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

3）更新参数的方法，**随机梯度下降法**：开始先随机生成一个 w，然后用下面的公式不断更新 w 的值，最终能够逼近真实结果。

$$w^{+} = w - \eta\frac{\partial Loss_{total}}{\partial w}$$

Part 03

运行环境准备

Install PYTORCH

https://pytorch.org/get-started/locally/



注意事项1：Pytorch对于仅支持CPU的电脑和支持GPU的电脑安装命令有所不同。

如果安装GPU版本，则需要根据主机的cuda版本来安装对应的torch，可以通过 nvidia-smi 查看cuda版本

```
C:\Users\Administrator>nvidia-smi
Sat Aug  2 16:03:18 2025
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 560.94                 Driver Version: 560.94         CUDA Version: 12.6     |
|-----------------------------------------+------------------------+----------------------+
| GPU  Name                     Driver-Model | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute M. |
|                                         |                        |               MIG M. |
|=========================================+========================+======================|
|   0  NVIDIA GeForce RTX 4070 ...   WDDM  |   00000000:01:00.0  On |                  N/A |
|  0%   38C    P8             20W /  285W |    3238MiB /  16376MiB |     21%      Default |
|                                         |                        |                  N/A |
+-----------------------------------------+------------------------+----------------------+

+-----------------------------------------------------------------------------------------+
| Processes:                                                                              |
|  GPU   GI   CI        PID   Type   Process name                              GPU Memory |
|        ID   ID                                                               Usage      |
|=========================================================================================|
```

注意事项2：确保本地 Python 版本要在 3.9 以上

Python 建议：3.11.1

Install PYTORCH



https://pytorch.org/get-started/locally/

```
C:\Users\Administrator>nvidia-smi
Sat Aug  2 16:03:18 2025
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 560.94                 Driver Version: 560.94         CUDA Version: 12.6     |
|-----------------------------------------+------------------------+----------------------+
| GPU  Name                     Driver-Model | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute M. |
|                                         |                        |               MIG M. |
|=========================================+========================+======================|
|   0  NVIDIA GeForce RTX 4070 ...  WDDM  |   00000000:01:00.0  On |                  N/A |
|  0%   38C    P8             20W /  285W |    3238MiB /  16376MiB |     21%      Default |
|                                         |                        |                  N/A |
+-----------------------------------------+------------------------+----------------------+

+-----------------------------------------------------------------------------------------+
| Processes:                                                                              |
|  GPU   GI   CI        PID   Type   Process name                              GPU Memory |
|        ID   ID                                                               Usage      |
|=========================================================================================|
```

NOTE: Latest PyTorch requires Python 3.9 or later.

PyTorch Build	Stable (2.7.1)		Preview (Nightly)	
Your OS	Linux	Mac	Windows	
Package	Pip	LibTorch	Source	
Language	Python		C++ / Java	
Compute Platform	CUDA 11.8	CUDA 12.6	CUDA 12.8	~~ROCm 6.3~~	CPU
Run this Command:	pip3 install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu126			

根据cuda的版本，直接根据 Run this Command中的命令进行安装即可

```
C:\Users\Administrator>pip3 install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu126
Looking in indexes: https://download.pytorch.org/whl/cu126
Collecting torch
  Downloading https://download.pytorch.org/whl/cu126/torch-2.7.1%2Bcu126-cp313-cp313-win_amd64.whl.metadata (27 kB)
Collecting torchvision
  Downloading https://download.pytorch.org/whl/cu126/torchvision-0.22.1%2Bcu126-cp313-cp313-win_amd64.whl.metadata (6.3
kB)
```

Install PYTORCH

https://pytorch.org/get-started/locally/



仅支持CPU的环境：

PyTorch Build	Stable (2.7.1)			Preview (Nightly)	
Your OS	Linux		Mac	Windows	
Package	Pip		LibTorch	Source	
Language	Python			C++ / Java	
Compute Platform	CUDA 11.8	CUDA 12.6	CUDA 12.8	~~ROCm 6.3~~	CPU
Run this Command:	pip3 install torch torchvision torchaudio				

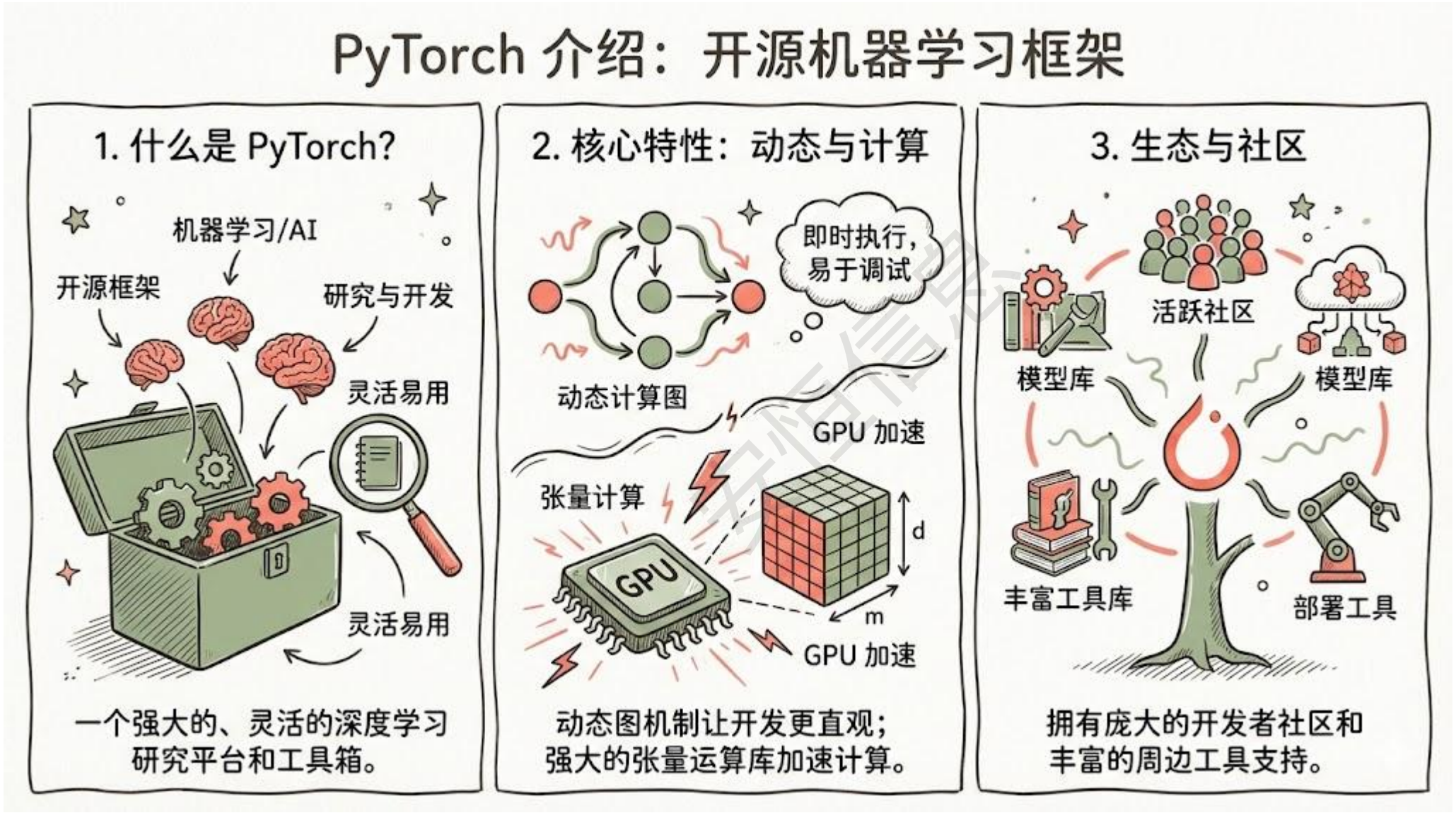
Run this command:	pip3 install torch torchvision torchaudio	机器仅 CPU 环境

练习 1-1：

验证电脑上是否成功安装 PyTorch!

```
C:\Users\> python
Python 3.11.1 (tags/v3.11.1:a7a450f, Dec  6 2022, 19:58:39) [MSC v.1934 64 bit (AMD64)] on
win32
Type "help", "copyright", "credits" or "license" for more information.
>>> import torch
>>> print(torch.__version__)
2.5.0+cpu
>>> # 成功安装!!
```

安恒信息

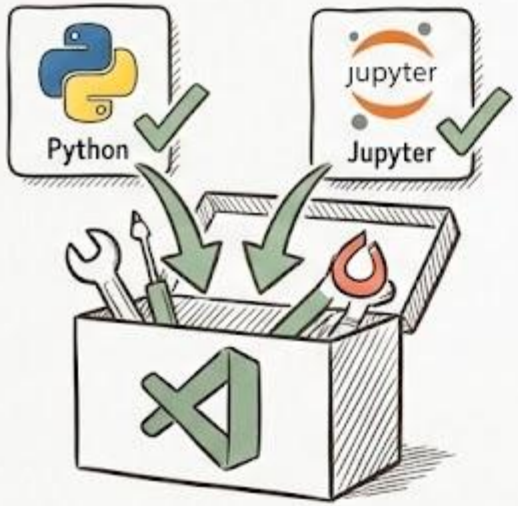

PyTorch 介绍：开源机器学习框架
1. 什么是 PyTorch?
机器学习/AI
开源框架
研究与开发
灵活易用
灵活易用
一个强大的、灵活的深度学习
研究平台和工具箱。
2. 核心特性：动态与计算
即时执行，
易于调试
动态计算图
GPU 加速
张量计算
GPU
d
m
GPU 加速
动态图机制让开发更直观；
强大的张量运算库加速计算。
3. 生态与社区
活跃社区
模型库
模型库
丰富工具库
部署工具
拥有庞大的开发者社区和
丰富的周边工具支持。

VS Code + Jupyter + PyTorch：搭建高效开发环境

(VS CODE + JUPYTER + PYTORCH: BUILDING AN EFFICIENT DEV ENVIRONMENT)

1. 准备工作：安装核心插件

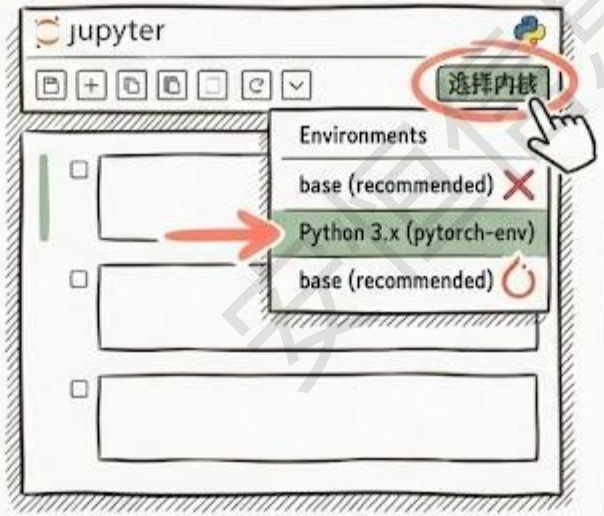
(PREPARATION: INSTALL CORE EXTENSIONS)



在 VS Code 扩展市场中安装 Python 和 Jupyter 插件。

2. 关键步骤：选择正确的内核

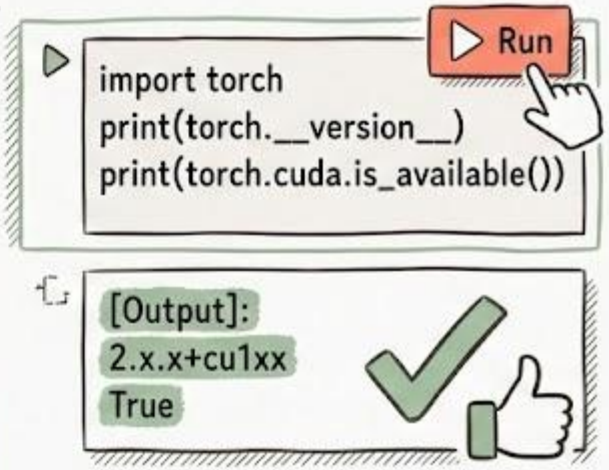
(CRITICAL STEP: SELECT THE RIGHT KERNEL)



点击内核选择器，指向你安装了 PyTorch 的那个环境。

3. 验证测试：运行 PyTorch 代码

(VERIFICATION: RUN PYTORCH CODE)



创建 .ipynb 文件，运行简单代码确认 PyTorch 加载成功。

Pytorch 代码运行




```python
## 导入模块
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score,confusion_matrix,classification_report

import torch
import torch.nn as nn                    ## nn 模块包含着torch 已经准备好的层，用于构建网络。
import torch.utils.data as Data          ## 这个功能包的作用是收集、打包数据，然后按照 batch 将数据分批喂给神经网络。
from torch.optim import  SGD,Adam        ## optim模块包含各种优化器
import matplotlib.pyplot as plt
import seaborn as sns
import hiddenlayer as hl                 ##  需要安装包（可视化工具，可以可视化网络）

pd.set_option('display.max_columns', None)
import warnings
warnings.filterwarnings("ignore")
```

垃圾邮件数据分类：https://archive.ics.uci.edu/ml/datasets/spambase

| SPAM E-MAIL DATABASE ATTRIBUTES (in .names format)

| 48 continuous real [0,100] attributes of type word_freq_WORD | = percentage of words in the e-mail that match WORD, | i.e. 100 * (number of times the WORD appears in the e-mail) / | total number of words in e-mail. A "word" in this case is any | string of alphanumeric characters bounded by non-alphanumeric | characters or end-of-string.

| 6 continuous real [0,100] attributes of type char_freq_CHAR | = percentage of characters in the e-mail that match CHAR, | i.e. 100 * (number of CHAR occurences) / total characters in e-mail

| 1 continuous real [1,...] attribute of type capital_run_length_average | = average length of uninterrupted sequences of capital letters

| 1 continuous integer [1,...] attribute of type capital_run_length_longest | = length of longest uninterrupted sequence of capital letters

| 1 continuous integer [1,...] attribute of type capital_run_length_total | = sum of length of uninterrupted sequences of capital letters | = total number of capital letters in the e-mail

| 1 nominal {0,1} class attribute of type spam | = denotes whether the e-mail was considered spam (1) or not (0), | i.e. unsolicited commercial e-mail.

---------- 解释 ----------

数据集包含57个特征，其中有48个特征是关键词出现的频率*100的取值，范围为[0,100],变量名用 word_freq_WORD 命名WORD表示该特征统计的词语。

6个特征为关键字符出现的频率*100的取值，范围为[0,100] 变量名用 char_freq_CHAR，

1个变量为 capital_run_length_average 表示大写字母不间断的平均长度。

1个变量 capital_run_length_longest 表示大写字母不间断的最大长度。

安恒信息

安恒信息
DAS-security 安全中国

谢谢观看

Thanks for watching



安恒信息

杭州安恒信息技术股份有限公司 XXX

www.dbappsecurity.com.cn